

Highcool Service Hub — AC Service Management System

Every planning document generated for this project, combined into one file — including all Mermaid diagrams (architecture, ERD, data flows, role/access, routing), pre-rendered as static images for PDF/print.

Contents

- 01 Conversation Summary
- 02 System Functions & Workflows
- 03 System Architecture
- 04 Folder Structure
- 05 Overall Design Plan
- 06 Database Design
- 07 Database Setup Guide (MySQL)
- 08 Implementation Plan — Step by Step
- 09 Diagrams
- 10 Tech Stack
- 11 Memory

Conversation Summary — AC Service Management System

This file summarizes the entire planning conversation for the AC Service Management System, in the order decisions were made. Use this as the master reference for "what was decided and why."

1. Original Requirement (Client's Raw Scenario)

An AC servicing company needs a system to manage:

- 1-year service agreements paid in full upfront
- Two service types: **Normal** (3 services/year) and **Hybrid** (4+ services/year)
- Automated SMS at 3 points: activation, reminder, completion
- Agreements identified by a serial number prefixed **AS-**
- Customers identified by **NIC** (search key) and **Phone** (unique ID)
- One customer can own multiple AC units, each with its own **AS-** number
- A calendar showing all scheduled jobs, clickable per day
- Technicians who search jobs by **AS-** number, execute the job, upload proof photos, and mark complete
- Postpone/reschedule capability for jobs
- Renewal each year (new **AS-** number issued, old one archived, not deleted)
- Cancellation → soft-delete/archive, not permanent deletion
- Customer loyalty/history tracking across years
- Technician performance reporting (jobs/day/week/month)
- Printable job cards
- Fully responsive web app (admin desktop use + technician mobile use)
- Cost-friendly build, reusing an existing **.lk** domain (later corrected — see below)

2. Requirements Gathering — Key Clarifications

QUESTION	ANSWER
Do technicians upload job photos from their phone?	Yes , directly via phone browser
Existing SMS gateway preference?	None — needed a recommendation
Current hosting setup?	cPanel, but PHP was not preferred

3. Stack & Hosting Decisions (in order decided)

1. **Frontend + Backend + Database stack:** React + Node.js/Express + MySQL — confirmed cost-friendly (all open-source), and matches the team's existing familiarity with Node/Python.
2. **Hosting provider:** Unlimited Hosting Lanka (UHL) — client's existing registrar/host.

3. **Hosting plan:** From UHL's menu (Budget/Business/Unlimited/Cloud Fast/WordPress/Windows/Email/MySQL/MSSQL/**Node.js-Python/LMS-CRM-APP**), the **Node.js/Python Hosting — Silver tier** (~18,500 LKR/year, 10GB NVMe SSD, 10 MySQL DBs, SSH, free SSL, daily backups) was selected — the only tier that explicitly guarantees Node.js support.
4. **SMS Gateway:** Compared **Notify.lk** (Rs 7,500 one-time fee + Rs 0.60/SMS) vs **Text.lk** (no setup/monthly fee, Rs 0.64–0.84/SMS depending on top-up). **Text.lk confirmed** — better fit given this project's naturally low SMS volume (no upfront cost to justify).
5. **Domain:** Initially assumed the client already owned a **.lk** domain from a prior registration with UHL. **Corrected later:** the domain has **not** been purchased yet — it will be registered (~4,500–5,000 LKR/year) **after** development is complete, just before go-live. Development/testing happens via local dev environment or cPanel's temporary URL in the meantime.

4. Other Confirmed Decisions

DECISION	ANSWER
Technician login	Individual account per technician (not shared) — enables accurate per-technician reporting
Reminder SMS lead time	1 day before the scheduled service date
Expected year-one scale	Not fixed — architecture comfortably handles small → several thousand customers without redesign
Photo upload limit	Max 5 photos per job, 5MB each
URL structure	Path-based, e.g. highcool.lk/admin , highcool.lk/technician — not subdomain-based
Frontend hosting	Not separate — React's production build is served as static files by the same Express app that runs the API, on the same hosting account

5. Documents Produced So Far

DOCUMENT	PURPOSE
AC_Service_Management_System_Plan.md	First full end-to-end plan (entities, workflows, phases)
Reordered_Scenario_Flow.md	Client's raw scenario reorganized into chronological order
System_Design.md	Technical design — architecture, ERD, API endpoints, folder structure
AC_System_Full_Design_Documentation.html	Consolidated HTML documentation with diagrams
AC_System_Budget_Proposal.docx	Client-facing budget — development cost range + infrastructure costs

6. Outstanding Open Items (as of this summary)

- Final SMS wording for the 3 templates (activation, reminder, completion)
- Job card exact layout — pending template from client
- Wireframes — pending from client

- Final development fee amount — still being decided internally (market range given: ~700,000–1,000,000 LKR)

7. Addendum — Wireframe Review (8 Hand-Drawn Sketches)

The client provided 8 photos of hand-drawn wireframes, which were reviewed and incorporated into the design. This revealed several structural refinements beyond the original plan:

- **3 roles, not 2:** Admin (total authority), System User (day-to-day office staff, no user/pricing management), Technician
- **AC unit details expanded:** Model, Brand, and two serial numbers (indoor + outdoor unit)
- **Service model replaced:** instead of one fixed "Normal or Hybrid" type, an agreement now allocates a **Normal count** + **H/P count** + a **Period** (30/60/90/120 days) — visits are scheduled from the combined count, and each visit is tagged Normal or H/P by the technician when performed
- **Completion approval flow added:** technician marking a job "Complete" now lands it in a **Job Complete Requests** queue; the Completion SMS only fires once Admin/System User confirms
- **Photo upload revised:** minimum 4 required (was previously max-5-only, no minimum)
- **Postpone/Cancel expanded:** postpone now requires a number of days + reason; cancel requires a reason; a new **Deleted Jobs** view (mistake corrections) is kept separate from **Job Cancellations** (customer-driven)
- **New Admin-only features:** Add Users (manage System User/Technician accounts), Add Price (default pricing per service type)
- **Job card:** Print and Download PDF (was print-only)
- **Calendar status colors confirmed:** Red = Active, Orange = Postponed, Green = Complete

All of 02 , 05 , 06 , 07 , 08 , 09 , and MEMORY.md were revised to reflect this. See MEMORY.md for the full list of corrections and what's still open (SMS wording, final job card visual styling, AS- number format confirmation, and a re-quote of the development fee given the added scope).

8. What This New File Set Covers

This conversation asked for a complete, implementation-ready markdown package, split into separate files:

1. 01_Conversation_Summary.md — this file
2. 02_System_Functions_And_Workflows.md — every function/module explained, how it works
3. 03_System_Architecture.md — backend → frontend → hosting, end to end
4. 04_Folder_Structure.md — full project folder structure
5. 05_Overall_Design_Plan.md — consolidated design plan
6. 06_Database_Design.md — schema, fields, relationships, ERD
7. 07_Database_Setup_Guide_MySQL.md — actual SQL to set up MySQL + sample queries
8. 08_Implementation_Plan_Step_By_Step.md — granular build plan, phase by phase, task by task
9. 09_Diagrams.md — all diagrams (architecture, ERD, data flow) in Mermaid format
10. 10_Tech_Stack.md — confirmed stack and why it aligns with Node.js + MySQL
11. MEMORY.md — condensed reference memory of this entire project/conversation

9. Addendum — Routing Revision (Root Path Reserved)

After reviewing a routing diagram, the client clarified: the root domain (`yourdomain.lk/`) is reserved for a **future company showcase/marketing site** (not part of this system, to be built later). **The entire system — both Admin and Technician sections — now sits under a single `/admin` base path** instead of splitting `/admin` and `/technician` at the root level. Express now serves the React build only under `/admin/*`; React Router uses `basename="/admin"`. Updated in `03_System_Architecture.md`, `09_Diagrams.md`, `04_Folder_Structure.md`, `05_Overall_Design_Plan.md`, and `MEMORY.md`.

System Functions & Workflows — AC Service Management System (Revised per Wireframes)

Every function in the system, what it does, what triggers it, and exactly how it works end to end. Updated after reviewing the client's hand-drawn wireframes.

1. Authentication & Roles (Revised — now 3 roles)

What it does: Controls who can access what.

Roles:

- **Admin** — total authority: everything System User can do, **plus** user management (add/update/delete users) and pricing management (Add Price)
- **System User** (office staff) — day-to-day operations: create/edit customers, agreements, jobs, assign technicians, renew, cancel/archive, view reports — **no** access to user management or pricing
- **Technician** — restricted: own assigned jobs only, search by **AS-** number, update job status, upload photos, postpone

How it works:

1. User logs in with phone/username + password (**POST /api/auth/login**)
 2. Backend verifies password (bcrypt), issues a JWT token containing **user_id** + **role**
 3. Frontend stores the token, attaches it to every subsequent API request
 4. Backend middleware checks the token's role on every protected route — Technicians blocked from admin/system-user routes; System Users blocked from user-management and pricing routes; only Admin passes those checks
-

2. Customer Registration — unchanged

Same as before: admin/system user enters details **after** full payment is confirmed. Fields: name, phone (unique), NIC (indexed), address, route.

3. AC Unit Registration (Expanded)

What it does: Represents one physical AC unit belonging to a customer — now with much richer detail per the wireframe.

Fields captured: Model, Brand, **Serial Number (Indoor unit)**, **Serial Number (Outdoor unit)** — since split ACs have two physical units, both are tracked separately.

How it works:

1. Each AC is added under a customer, with Model/Brand/dual serials

2. One customer can have multiple AC units — each gets its own agreement and `AS-` number
 3. Entering a customer's NIC always surfaces **all** AC units linked to them
 4. "+ Add AC" button lets admin/system user add another AC unit to the same customer without leaving the form
-

4. Agreement Creation & AS- Numbering — numbering unchanged, service model revised

What it does: Creates the 1-year service contract tied to one AC unit.

How it works:

1. Admin/system user fills the Create Job form: customer details, AC details, then Agreement data
 2. `numberingService.js` generates the next `AS-` serial — global auto-increment, never reused
 3. **Instead of picking one fixed "Normal" or "Hybrid" type**, the form now captures:
 - **Normal count** — how many Normal-type visits this agreement includes
 - **H/P count** — how many Hybrid-type visits this agreement includes
 - **Period** — days between each visit (30 / 60 / 90 / 120, selectable presets)
 - **Price** — pulled from the Pricing table by default, editable per agreement
 4. `start_date` = today, `end_date` = start_date + 1 year, status = `active`
-

5. Service Type Model (Revised — replaces old Normal-vs-Hybrid toggle)

Old model: One fixed type per agreement (Normal = 3 visits/year, Hybrid = 4+ visits/year).

New model (per wireframe): An agreement can mix both types — e.g. 2 Normal visits + 2 H/P visits, spaced 90 days apart. The **total number of visits generated = Normal count + H/P count**. Each individual visit isn't pre-labeled Normal or H/P at creation — instead, the **technician tags each visit** as Normal or H/P when they actually perform it (seen on the technician's "Inside Job" screen as a Normal/H-P selector). This tagging decrements the corresponding count.

Example: Normal=2, H/P=2, Period=90 days → system schedules 4 visits, 90 days apart. As each visit happens, the technician marks it Normal or H/P.

6. Automatic Job Scheduling (Revised)

What it does: Generates the year's service visit dates the moment an agreement is created.

How it works:

1. `schedulerService.js` runs right after agreement creation
 2. Creates `(normal_count + hp_count)` rows in `jobs`, spaced `period_days` apart, starting from `start_date`
 3. Each job starts with status `scheduled`, no technician assigned yet, and `service_type_used` left blank (set later by the technician)
-

7. Pricing Management (New — Admin only)

What it does: Lets Admin set/update default prices for Normal and H/P service types.

How it works:

1. Admin-only screen ("Add Price")
 2. `pricing` table stores one row per service type with a price
 3. When creating a new agreement, the form pre-fills `price` from this table (still editable per individual agreement, since some customers may negotiate differently)
-

8. User Management (New — Admin only)

What it does: Lets Admin add, update, or deactivate System User and Technician accounts.

How it works:

1. Admin-only screen ("Add Users" with manage: Add/Update/Delete)
 2. Creates rows in `users` with the appropriate role
 3. "Delete" is implemented as deactivation (`active = FALSE`), not a hard delete, to preserve historical job/report links to that user
-

9. SMS Automation (3 Templates) — trigger point refined

What it does: Sends automatic SMS at 3 points in the lifecycle, via **Text.Ik**.

TRIGGER	TEMPLATE TYPE	WHEN IT FIRES
Agreement created	Activation	Immediately after registration
1 day before scheduled job	Reminder	Daily cron job checks tomorrow's jobs
Job confirmed complete	Completion	Fires when admin confirms the technician's completion (see function 12 — Job Complete Requests), not the instant the technician taps Complete

How it works:

1. `smsService.js` wraps the Text.Ik REST API
 2. Every send is logged in `sms_logs`
 3. Reminder driven by a daily cPanel cron job (`reminderCron.js`)
-

10. Calendar View (Expanded per wireframe)

What it does: Shows every scheduled job visually, by date, with status color-coding.

Status colors (confirmed from wireframe):

-  **Red** = Active/Scheduled
-  **Orange** = Postponed

-  **Green** = Complete

How it works:

1. Top-level dashboard shows search (by id/phone/AS-), "+ Create Job", "+ Renew Job" buttons, a Job Calendar widget, and a Job Complete Requests widget
 2. Detailed Calendar screen: search bar, Month/Year filter, status counters (Active / Postponed / Complete, each with a count), and a month grid
 3. Each day cell shows colored dots representing that day's jobs by status
 4. Clicking a day ("Inside day") expands to a list: AC count, customer name, **AS-** number, status badge with icon (● Active, ! Postponed, ✓ Complete)
-

11. Technician Assignment — unchanged

Admin/system user selects a job, assigns a technician via `PATCH /api/jobs/:id/assign` .

12. Job Complete Requests (New)

What it does: A review queue — technicians marking a job "complete" doesn't finalize it instantly; it goes into a queue for Admin/System User to confirm.

How it works:

1. Technician taps Complete → job status becomes `completed` , but `admin_confirmed = FALSE`
 2. Job appears in the **Job Complete Requests** screen (admin-facing)
 3. Admin/system user reviews (can check photos, comments) and confirms
 4. Confirming sets `admin_confirmed = TRUE` — **this is what actually triggers the Completion SMS**
-

13. Job Card — Print & PDF Download (Expanded)

What it does: Generates a reference sheet for a specific job/visit.

How it works:

1. `GET /api/jobs/:id/print` renders a print-friendly view
 2. **New:** a "Download PDF" button alongside Print, so the job card can be saved/shared digitally, not just printed
 3. Shows customer details, address, route, **AS-** number, AC info (model/brand/serials), and comments
-

14. Technician Job Execution (Revised: photo minimum + type tagging)

What it does: The technician's entire on-site workflow, from their phone browser.

How it works:

1. Technician logs in, sees Today's Jobs or searches by **AS-** number, ID, or phone
2. Opens the job ("Inside Job" view) — shows AS- number, a "Customer details" button for quick reference
3. Taps **Start** → job status becomes `in_progress`

4. Uploads photos via phone camera — **minimum 4 required, maximum 5**, 5MB each — job cannot be marked complete below the 4-photo minimum
 5. Selects the visit type — **Normal or H/P** — via a selector on this screen (this is what sets `service_type_used`)
 6. Adds comments if needed
 7. Taps **Complete Job** → status becomes `completed` , `admin_confirmed = FALSE` (goes to the Job Complete Requests queue — function 12)
 8. If the customer asks to reschedule → **Postpone** instead (function 15)
-

15. Postpone / Cancel (Expanded per wireframe)

Postpone:

1. Requires **number of days** to postpone by, **plus a reason**
2. `postponed_from` stores the original date, `postpone_days` + `postpone_reason` logged
3. `scheduled_date` shifts forward by the specified number of days
4. Status becomes `postponed` (shows orange on the calendar)

Cancel:

1. Requires a **reason/comment**
 2. Status becomes `cancelled` , reason stored in `cancel_reason`
 3. Appears in the **Job Cancellations** view (distinct from Deleted Jobs — see function 17)
-

16. Search & Filtering (NIC / Phone / AS-) — unchanged

Universal lookup, returns full linked history when searching by NIC or Phone; direct jump when searching by `AS-` .

17. Deleted Jobs vs Job Cancellations (New — two distinct admin views)

- **Deleted Jobs** — jobs soft-deleted at the job level (`is_deleted = TRUE`) — typically for correcting mistakes (e.g. a duplicate or wrongly created job), not a customer-driven cancellation
- **Job Cancellations** — jobs the customer actually cancelled (`status = 'cancelled'` , with a reason logged)

Both are admin-facing list views, both are soft-deletes (never hard-deleted from the database).

18. Renewal (Revised flow, same numbering logic)

What it does: Extends a customer's service for another year.

How it works:

1. Admin/system user searches by ID/ `AS-` /Phone ("Renew AC" screen)
2. This loads the **same Update/Job form**, pre-filled with the existing AC and agreement details
3. Admin adjusts Model/Brand/Serials/Normal count/H-P count/Period/Price as needed for the new year

4. Taps **Renew** → creates a **new** agreement row with a **new** **AS-** number, **parent_agreement_id** links to the old one, old marked **renewed**
 5. Job scheduling (function 6) runs again for the new year
-

19. Cancellation & Archiving (Agreement-level) — unchanged

POST /api/agreements/:id/cancel soft-deletes the whole agreement (status → **cancelled**), visible in Archive, never hard-deleted.

20. Customer History & Loyalty Tracking — unchanged

Customer profile shows every agreement ever linked to that NIC/Phone, **created_at** marks loyalty start, renewal chains show year-over-year relationships.

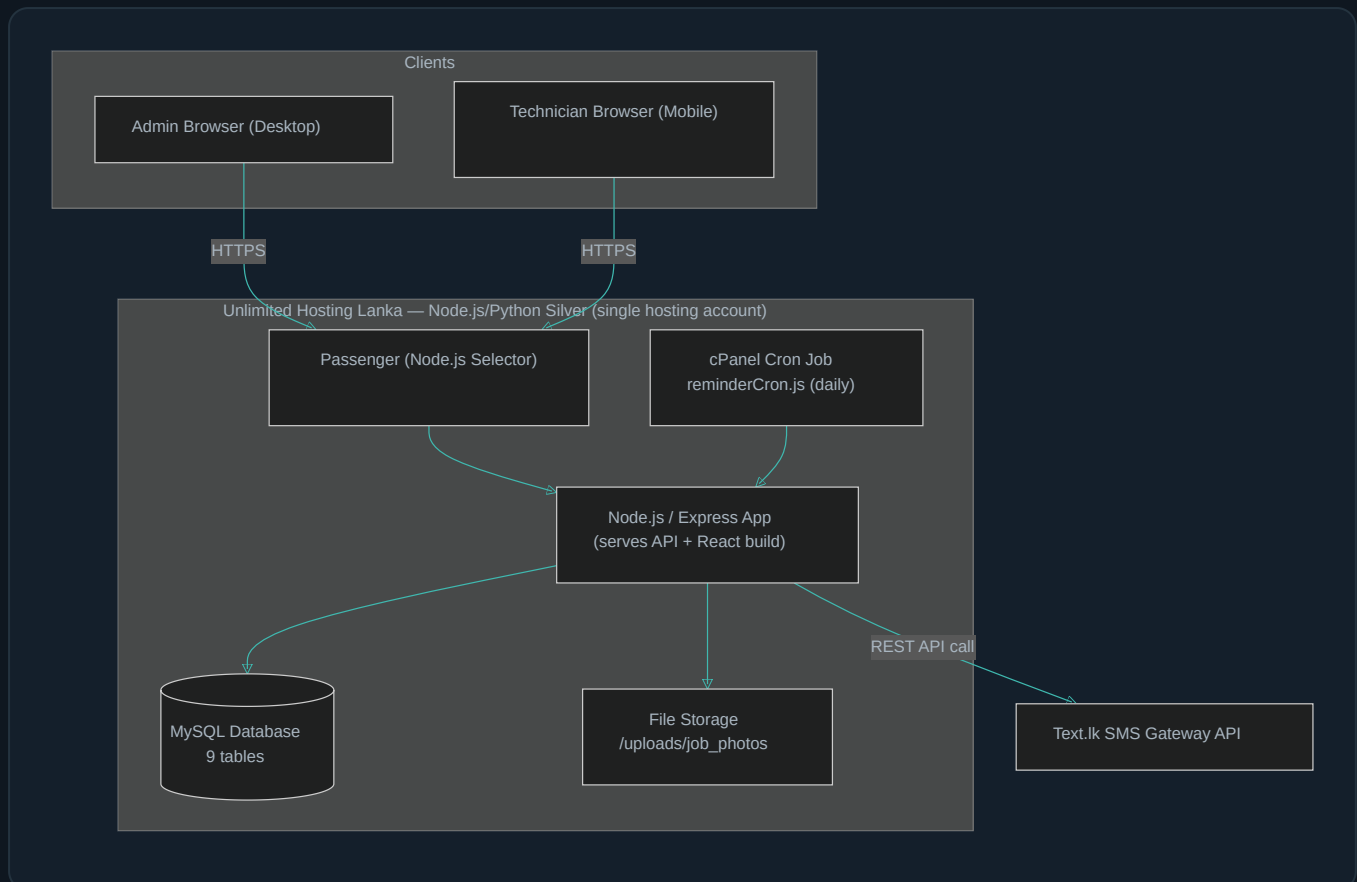
21. Reporting (Technician Performance) — refined

Counts only **admin-confirmed** completions (not just technician-marked-complete), grouped by day/week/month, filtered by technician.

System Architecture — Backend → Frontend → Hosting

This describes exactly how the system runs, physically and logically, from the database up to the browser.

1. High-Level Architecture



Key point: everything — API, frontend, and database — runs on **one hosting account**. There is no separate frontend host, no separate database server, and no separate application server. This keeps cost and operational complexity minimal.

2. Backend Layer (Node.js / Express)

- Express serves two things from the same process:
 - /api/*** routes — all business logic (auth, customers, agreements, jobs, sms, reports)
 - /admin/*** — the React production build's static files (`index.html`, JS, CSS) — the entire system (Admin + Technician sections) lives under this one path prefix, since the root domain is reserved for a future showcase site

```
// app.js – the core of how backend + frontend coexist in one app
app.use('/api', apiRouter);
app.use('/admin', express.static('client/build'));
app.get('/admin/*', (req, res) => res.sendFile(path.join(__dirname,
'client/build/index.html')));
```

```
// Root path: reserved for a future showcase site.  
// For now, simplest options are a placeholder page, or a redirect straight into the system:  
app.get('/', (req, res) => res.redirect('/admin'));
```

- Backend talks to **MySQL** on `localhost` (same server) via `mysql2` or an ORM (Sequelize/Knex)
- Backend talks to **Text.lk** over HTTPS for SMS sending
- Backend reads/writes job photos to a local `/uploads/job_photos/` folder (outside `public_html`, served only via an authenticated route)

3. Frontend Layer (React)

- **Built locally** (or in CI) via `npm run build` → produces a static `build/` folder
- That folder is uploaded into the Node app's directory and served by Express — **no separate hosting needed**
- **Revised routing decision:** the root domain (`yourdomain.lk/`) is reserved for a future company showcase/marketing site (not part of this system, to be built later). **The entire system — both Admin and Technician sections — is mounted under a single `/admin` base path** for now.
- **React Router** uses a `basename="/admin"` so all internal routes sit under that prefix automatically:

yourdomain.lk/

(Reserved for future showcase
site — not built yet)

Login Page

/admin/dashboard

/admin/customers

/admin/calendar

/admin/archive

/admin/reports

yourdomain.lk/admin/

/admin/job-complete-requests

/admin/deleted-jobs

/admin/cancellations

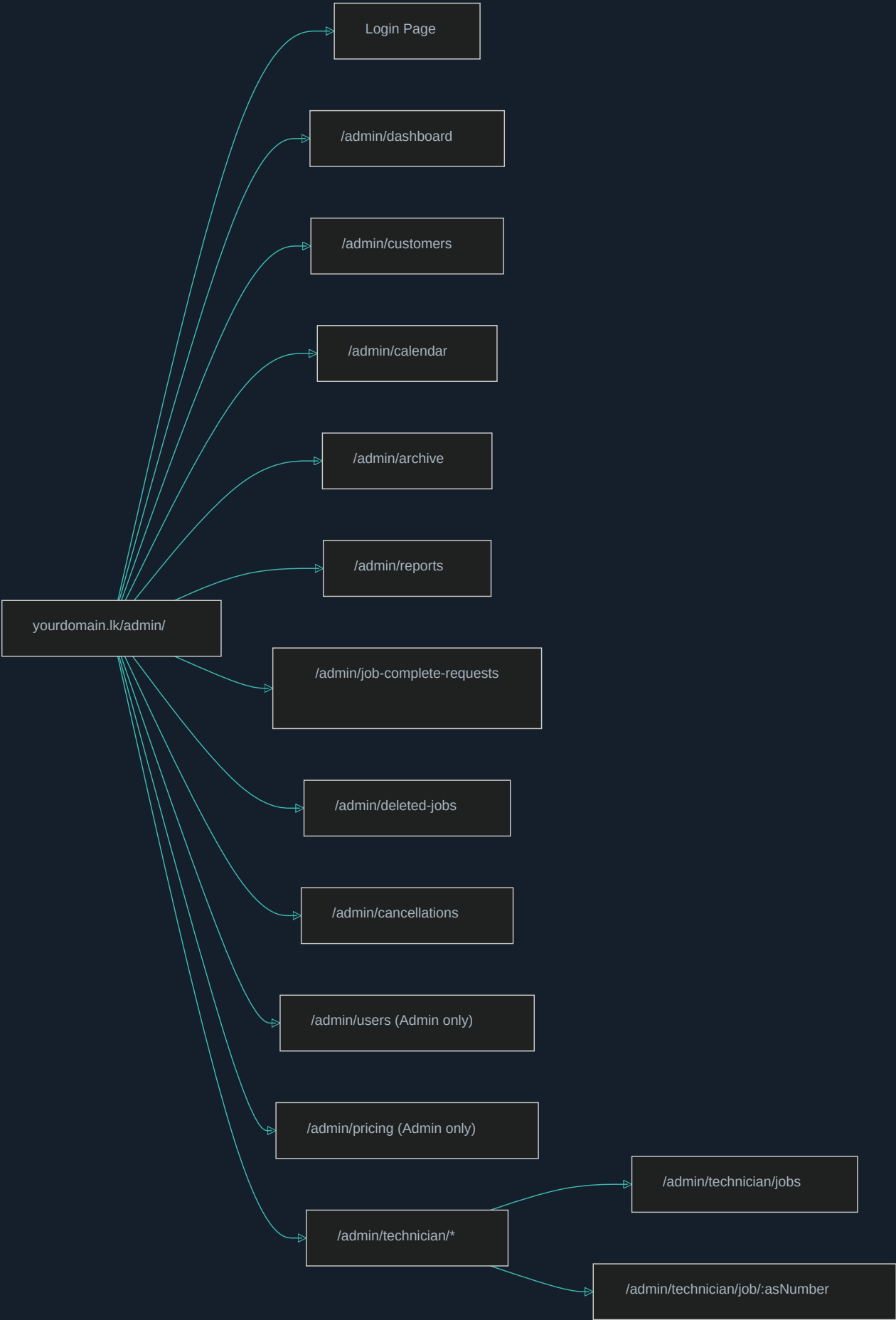
/admin/users (Admin only)

/admin/pricing (Admin only)

/admin/technician/*

/admin/technician/jobs

/admin/technician/job/:asNumber



- Role-based route guards on the frontend (redirect if wrong role) — but the **real** security is backend-side JWT role checks on every API call, since URLs alone are not protection

4. Database Layer (MySQL)

- Single MySQL database, same server as the app (no networking config needed)
- 9 tables: `customers`, `ac_units`, `agreements`, `jobs`, `job_photos`, `sms_logs`, `users`, `pricing` — see `06_Database_Design.md` (revised per client wireframes: 3 roles, Normal/H-P count model, completion approval flow)
- Full schema in `06_Database_Design.md`, setup SQL in `07_Database_Setup_Guide_MySQL.md`

5. Hosting Layer (cPanel specifics)

How cPanel runs a Node.js app:

- cPanel uses **CloudLinux's Node.js Selector**, backed by **Phusion Passenger**, to supervise the Node process
- Under *Setup Node.js App* in cPanel, you configure: Node version, application root, application URL, startup file (`app.js`), and environment variables (DB credentials, JWT secret, Text.lk API key)
- Passenger assigns an internal port automatically and proxies the domain's traffic straight to the Express app — no manual port management

Domain routing:

- Once the `.lk` domain is purchased and pointed to this hosting account (post-development, before go-live), it's attached as the app's Application URL
- Requests to `highcool.lk/admin`, `highcool.lk/admin/dashboard`, `highcool.lk/admin/technician/job/AS-00123` all hit the Express `/admin/*` catch-all, and React Router (with `basename="/admin"`) renders the correct screen client-side
- `highcool.lk/` itself is reserved for the future showcase site — for now it can either show a simple placeholder or redirect straight into `/admin`

Deployment workflow:

1. Push code via cPanel's Git Version Control (or SFTP/File Manager)
2. Click **"Run NPM Install"** in the Node.js App interface
3. Click **"Restart"** to reload the app
4. For frontend-only changes: rebuild React locally, re-upload `build/`, restart

Cron (SMS reminders):

```
0 8 * * * /usr/bin/node /home/username/ac-service-app/src/jobs/reminderCron.js
```

Runs daily at 8am, checks for jobs due tomorrow, triggers reminder SMS via Text.lk.

SSL: Free Let's Encrypt certificate via cPanel, auto-renews, activated once the domain is attached.

6. Security Model Summary

LAYER	PROTECTION
Passwords	bcrypt hashing, never stored in plaintext
Sessions	JWT tokens, role embedded and checked server-side
Routes	Role-guarded on frontend (UX) and backend (actual enforcement)
Photos	Stored outside public web root, served only via authenticated route
Transport	HTTPS enforced via free Let's Encrypt SSL
Data integrity	Soft-deletes only (agreements never hard-deleted) — full audit trail preserved

Folder Structure — AC Service Management System

Single-repo (or single-app-root) structure, since backend and frontend deploy together to one cPanel Node.js app.

```

ac-service-app/                                # Application root on cPanel
|
├─ server/                                     # Backend (Node.js / Express)
|   └─ app.js                                 # Entry point – configures Express, serves API + React
build
|   └─ config/
|       └─ db.js                             # MySQL connection config (reads from env vars)
|   └─ routes/
|       └─ auth.routes.js
|       └─ customers.routes.js
|       └─ agreements.routes.js
|       └─ jobs.routes.js
|       └─ sms.routes.js
|       └─ reports.routes.js
|       └─ users.routes.js                   # NEW – Admin only: manage system_user/technician
accounts
|   └─ pricing.routes.js                     # NEW – Admin only: Normal/H-P pricing
|   └─ controllers/
|       └─ auth.controller.js
|       └─ customers.controller.js
|       └─ agreements.controller.js
|       └─ jobs.controller.js
|       └─ sms.controller.js
|       └─ reports.controller.js
|       └─ users.controller.js               # NEW
|       └─ pricing.controller.js            # NEW
|   └─ models/
|       └─ customer.model.js
|       └─ acUnit.model.js
|       └─ agreement.model.js
|       └─ job.model.js
|       └─ jobPhoto.model.js
|       └─ smsLog.model.js
|       └─ user.model.js
|       └─ pricing.model.js                 # NEW
|   └─ services/
|       └─ smsService.js                     # Text.lk API wrapper + template rendering
|       └─ schedulerService.js               # Generates the year's job dates on agreement creation
|       └─ numberingService.js               # AS- number generator
|   └─ jobs/
|       └─ reminderCron.js                   # Daily cron script – checks jobs due tomorrow
|   └─ middleware/

```

```

| | └─ auth.middleware.js      # JWT verification
| | └─ role.middleware.js      # Role-based route guarding (admin / system_user /
technician)
| └─ uploads/
|   └─ job_photos/            # Stored technician photos (not publicly browsable)
|   └─ package.json
|
└─ client/                    # Frontend (React)
  └─ public/
    └─ index.html
  └─ src/
    └─ pages/
      └─ admin/
        └─ Dashboard.jsx
        └─ CustomerSearch.jsx
        └─ CustomerProfile.jsx
        └─ NewAgreement.jsx    # Create Job – customer + AC + agreement
(Normal/H-P/Period)
        └─ RenewAgreement.jsx  # Renew AC – search then reuses NewAgreement-
style form
        └─ Calendar.jsx        # month/year filter, status color-coded day
cells
        └─ JobSlot.jsx          # "Job slot inside" – job detail,
postpone/cancel/print/PDF
        └─ JobCard.jsx          # print + PDF download
        └─ JobCompleteRequests.jsx # NEW – admin approval queue for technician
completions
        └─ DeletedJobs.jsx      # NEW – soft-deleted jobs view
        └─ JobCancellations.jsx # NEW – cancelled jobs view (distinct from
Deleted Jobs)
        └─ Archive.jsx          # cancelled agreements
        └─ Reports.jsx
        └─ SmsTemplates.jsx
        └─ AddUsers.jsx         # NEW – Admin only: manage
system_user/technician accounts
        └─ AddPrice.jsx         # NEW – Admin only: manage Normal/H-P default
pricing
        └─ technician/
          └─ Login.jsx
          └─ TodayJobs.jsx
          └─ JobSearch.jsx
          └─ JobDetail.jsx      # start / photo upload (min 4, max 5) / Normal-or-H-P
tag / complete / postpone
        └─ components/
          └─ SearchBar.jsx
          └─ CalendarGrid.jsx   # status color-coded (red/orange/green)
          └─ JobCardPrint.jsx   # print + PDF download
          └─ PhotoUploader.jsx  # enforces min 4 / max 5 photos client-side
        └─ api/
          └─ customers.api.js
          └─ agreements.api.js
          └─ jobs.api.js

```

```

| | | └─ sms.api.js
| | | └─ reports.api.js
| | | └─ users.api.js      # NEW
| | | └─ pricing.api.js    # NEW
| | └─ App.jsx             # React Router route definitions (basename="/admin")
| └─ index.js
└─ build/                  # Production build output (uploaded to server/, served
by Express)
  └─ package.json
  |
└─ docs/                  # This entire markdown documentation set
  └─ 01_Conversation_Summary.md
  └─ 02_System_Functions_And_Workflows.md
  └─ 03_System_Architecture.md
  └─ 04_Folder_Structure.md
  └─ 05_Overall_Design_Plan.md
  └─ 06_Database_Design.md
  └─ 07_Database_Setup_Guide_MySQL.md
  └─ 08_Implementation_Plan_Step_By_Step.md
  └─ 09_Diagrams.md
  └─ 10_Tech_Stack.md
  └─ MEMORY.md
  |
└─ .env                   # Environment variables (DB creds, JWT secret, Text.lk
API key)
└─ .gitignore
└─ README.md

```

Notes on Structure

- **Why one repo, not two:** since both frontend and backend deploy to the same cPanel Node.js app, keeping them in one project (`server/` + `client/`) simplifies deployment — one Git push, one restart.
- `client/build/` is generated, not hand-written — regenerate with `npm run build` inside `client/` whenever frontend code changes, then it gets served by `server/app.js` .
- `uploads/job_photos/` lives inside `server/` , not `client/public/` or any public-facing folder — this is what keeps photos from being directly browsable by URL guessing.
- `.env` holds all secrets and is never committed to Git — cPanel's Node.js App interface also lets you set these as environment variables directly in its UI.
- **Routing note (revised):** the root domain is reserved for a future company showcase site — this entire application (Admin + Technician) is mounted under `/admin` via Express (`app.use('/admin', ...)`) and React Router's `basename="/admin"` , so no folder here needs restructuring, only the routing configuration in `app.js` and `App.jsx` .

Overall Design Plan — AC Service Management System

(Revised per Wireframes)

A single consolidated view of the system design. Detailed breakdowns live in the other numbered files — this is the "read this first" overview. This version incorporates the client's hand-drawn wireframes.

1. Purpose

A web-based system for an AC servicing company to manage 1-year service agreements, automated SMS notifications, technician scheduling and job execution (with photo proof), renewals, cancellations, customer history, and performance reporting — fully responsive for admin desktop use and technician mobile use.

2. Confirmed Tech Stack

LAYER	CHOICE
Frontend	React
Backend	Node.js + Express
Database	MySQL
Hosting	Unlimited Hosting Lanka — Node.js/Python Silver plan
SMS Gateway	Text.lk
File Storage	Server disk (cPanel)

(Full reasoning in `10_Tech_Stack.md`.)

3. User Roles (Revised — now 3, not 2)

ROLE	ACCESS
Admin	Total authority — everything below, plus user management (add/update/delete users) and pricing management
System User (office staff)	Day-to-day operations — customers, agreements, jobs, technician assignment, renewals, cancellations, reports. No user management or pricing access
Technician	Restricted — own assigned jobs, search by <code>AS-</code> , status updates, photo upload (min 4/max 5), tag visit as Normal/H-P, postpone

Each technician has an **individual login**.

4. Core Entities (Revised)

- Customer** — identified by NIC (search key) and Phone (unique ID)

- **AC Unit** — one or more per customer; now includes **Model, Brand, Indoor Serial, Outdoor Serial**
- **Agreement** — the **AS-** numbered 1-year contract, one per AC unit; now tracks **Normal count + H/P count + Period(days) + Price** instead of one fixed service type
- **Job** — an individual scheduled visit; now tracks **which type it turned out to be (Normal/H-P)**, an **admin-confirmation flag**, postpone days, cancel reason, and comments
- **Job Photo** — proof-of-work images, min 4/max 5 per job
- **SMS Log** — record of every message sent
- **Pricing** — Admin-managed default price per service type (**new**)
- **User** — admin/system user/technician login accounts

(Full schema in **06_Database_Design.md**.)

5. Core Modules / Screens (Revised)

Admin (14 screens, up from 9): Dashboard, Customer Search, Customer Profile, New Agreement (Create Job), Renew Agreement, Calendar, Job Slot (detail/postpone/cancel/print/PDF), Job Card print, **Job Complete Requests (new)**, **Deleted Jobs (new)**, **Job Cancellations (new)**, Archive, Reports, SMS Templates, **Add Users (new, Admin-only)**, **Add Price (new, Admin-only)**

Technician (4 screens): Login, Today's Jobs, Job Search, Job Detail (start/photo upload/type-tag/complete/postpone)

6. Key Business Rules (Revised)

1. Data is only entered into the system **after** full payment is confirmed
2. Every AC unit gets its own **AS-** number, serial, never reused
3. **New:** An agreement allocates a **mix** of Normal and H/P visits (counts) rather than one fixed type, spaced by a chosen **Period** (30/60/90/120 days)
4. **New:** Each visit's actual type (Normal or H/P) is tagged by the technician when performed, not pre-assigned
5. Renewal issues a **new AS-** number, links to the old one via **parent_agreement_id** — nothing is overwritten
6. Cancellation is **always** a soft-delete (archive/cancellation view), never a hard delete
7. **New:** Deleted Jobs (mistake corrections) and Job Cancellations (customer-driven) are tracked as **two distinct views**
8. Searching by NIC or Phone always surfaces **every** AC/agreement linked to that person
9. SMS fires automatically at 3 points: activation, 1-day-before reminder, and completion — **completion SMS now fires only after Admin/System User confirms** the technician's completion (Job Complete Requests)
10. Photos: **min 4, max 5** per job, 5MB each, uploaded directly from the technician's phone
11. **New:** Pricing is centrally managed by Admin, applied as a default when creating agreements, still editable per agreement

7. Deployment Model — Revised (root path reserved)

One hosting account (cPanel, Node.js/Python Silver) runs everything. **New:** the root domain (**yourdomain.lk/**) is reserved for a future company showcase site (not built yet, later phase) — the entire system (Admin + Technician) is mounted under **/admin** instead. (Full architecture in **03_System_Architecture.md**.)

8. Cost Summary — unchanged

See `AC_System_Budget_Proposal.docx` for the full breakdown (hosting, domain, SMS).

9. Build Sequence

10 phases — see `08_Implementation_Plan_Step_By_Step.md`, updated to include Pricing and User Management (Phase 1), and Job Complete Requests / Deleted Jobs / Job Cancellations (Phase 3/7).

10. Still Open / Pending From Client

- Final SMS template wording (activation, reminder, completion)
 - Job card exact layout — wireframe gives structure, final visual styling still to confirm
 - Final development fee (market range ~700,000–1,000,000 LKR, not yet finalized)
 - AS- number format — wireframe examples show a longer numeric string (e.g. "AS260718..."); current design keeps the simpler `AS-00001` serial format unless the client confirms a specific date-encoded format is required
-

Database Design — AC Service Management System (Revised per Wireframes)

MySQL, **9 tables** (was 7 — added **pricing** ; expanded fields per client wireframes). Everything keys off the Agreement (**AS-** number), cross-linked to a Customer via NIC/Phone.

Revision note: This replaces the earlier 7-table version after reviewing the client's hand-drawn wireframes. Key changes: 3 roles (not 2), richer AC unit details (model/brand/dual serials), agreements now track separate Normal/H-P visit counts + a period instead of one fixed service type, jobs track which type each visit actually was, a completion-approval flow, and a pricing table.

Entity Relationship Diagram

CUSTOMERS		
int	id	PK
varchar	name	
varchar	phone	UK
varchar	nic	
text	address	
varchar	route	
datetime	created_at	

PRICING			
int	id	PK	
enum	service_type		normal hp
decimal	price		
datetime	updated_at		

AC_UNITS		
int	id	PK
int	customer_id	FK
varchar	model	
varchar	brand	
varchar	serial_indoor	
varchar	serial_outdoor	
text	install_notes	

AGREEMENTS			
int	id	PK	
varchar	agreement_no	UK	AS-00001
int	customer_id	FK	
int	ac_unit_id	FK	
int	normal_count		
int	hp_count		
int	period_days		30/60/90/120
decimal	price		
date	start_date		
date	end_date		
decimal	amount_paid		
enum	status		active expired cancelled renewed
int	parent_agreement_id	FK	
int	created_by	FK	
datetime	created_at		

USERS			
int	id	PK	
varchar	name		
varchar	phone		
enum	role		admin system_user technician
varchar	password_hash		
boolean	active		

JOBS			
int	id	PK	
int	agreement_id	FK	
date	scheduled_date		
enum	status		scheduled in_progress completed postponed cancelled
int	technician_id	FK	
enum	service_type_used		normal hp
boolean	admin_confirmed		
boolean	is_deleted		
date	postponed_from		
int	postpone_days		
text	postpone_reason		
text	cancel_reason		
text	comments		
datetime	completed_at		
text	notes		
datetime	created_at		

SMS_LOGS	
----------	--

linked via customer_id

default price reference

one AC = one active agreement

assigned_to (technician_id)

schedules

parent_agreement_id (renewal chain)

triggers

has

FIELD	TYPE	NOTES
model	VARCHAR(100)	new
brand	VARCHAR(100)	new
serial_indoor	VARCHAR(100)	new — indoor unit serial
serial_outdoor	VARCHAR(100)	new — outdoor unit serial
install_notes	TEXT	optional

agreements — service model replaced

FIELD	TYPE	NOTES
id	INT, PK, AUTO_INCREMENT	
agreement_no	VARCHAR(20), UNIQUE	format AS-00001
customer_id	INT, FK → customers.id	
ac_unit_id	INT, FK → ac_units.id	
normal_count	INT	new — number of Normal visits allocated this year
hp_count	INT	new — number of H/P (Hybrid) visits allocated this year
period_days	INT	new — days between visits: 30 / 60 / 90 / 120
price	DECIMAL(10,2)	new — agreed price for this agreement (pulled from pricing table by default, editable per agreement)
start_date	DATE	
end_date	DATE	start_date + 1 year
amount_paid	DECIMAL(10,2)	
status	ENUM('active','expired','cancelled','renewed')	
parent_agreement_id	INT, FK → agreements.id, NULLABLE	links renewal chain
created_by	INT, FK → users.id	
created_at	DATETIME	

Removed: the old single **service_type** `ENUM('normal','hybrid')` field — replaced by **normal_count** + **hp_count**, since an agreement can now mix both types (e.g. 2 Normal + 2 H/P visits) rather than being locked to one type.

jobs — expanded per wireframe

FIELD	TYPE	NOTES
id	INT, PK, AUTO_INCREMENT	
agreement_id	INT, FK → agreements.id	
scheduled_date	DATE	
status	ENUM('scheduled','in_progress','completed','postponed','cancelled')	
technician_id	INT, FK → users.id, NULLABLE	
service_type_used	ENUM('normal','hp'), NULLABLE	new — set by technician when they execute the visit; decrements the corresponding count on the agreement
admin_confirmed	BOOLEAN, DEFAULT FALSE	new — powers the "Job Complete Requests" queue; technician marking complete doesn't finalize it until admin confirms
is_deleted	BOOLEAN, DEFAULT FALSE	new — powers the "Deleted Jobs" view (soft-delete at job level, separate from a full cancellation)
postponed_from	DATE, NULLABLE	
postpone_days	INT, NULLABLE	new — number of days postponed, as shown in wireframe
postpone_reason	TEXT, NULLABLE	
cancel_reason	TEXT, NULLABLE	new — required when a job is cancelled
comments	TEXT, NULLABLE	new — general notes/comments thread on the job
completed_at	DATETIME, NULLABLE	
notes	TEXT	
created_at	DATETIME	

job_photos — unchanged structure, new limit

FIELD	TYPE	NOTES
id	INT, PK, AUTO_INCREMENT	
job_id	INT, FK → jobs.id	
photo_path	VARCHAR(255)	

FIELD	TYPE	NOTES
uploaded_at	DATETIME	
uploaded_by	INT, FK → users.id	

Enforced at application level: minimum **4** photos required before a job can be marked complete, maximum **5** photos per job, 5MB each.

sms_logs — unchanged

FIELD	TYPE	NOTES
id	INT, PK, AUTO_INCREMENT	
customer_id	INT, FK → customers.id	
job_id	INT, FK → jobs.id, NULLABLE	
template_type	ENUM('activation','reminder','completion')	
message	TEXT	
sent_at	DATETIME	
status	VARCHAR(50)	

pricing — new table

FIELD	TYPE	NOTES
id	INT, PK, AUTO_INCREMENT	
service_type	ENUM('normal','hp')	
price	DECIMAL(10,2)	
updated_at	DATETIME	

Powers the Admin-only "Add Price" screen — sets default pricing per service type, which pre-fills the **agreements.price** field (still editable per individual agreement).

Relationship Summary

- customers** 1—many **ac_units**
- customers** 1—many **agreements** (across all their ACs and renewal years)
- ac_units** 1—1 **agreements** (an AC has one *active* agreement at a time; history via renewal chain)
- agreements** 1—many **jobs**
- agreements** self-referencing via **parent_agreement_id** (renewal history chain)
- jobs** 1—many **job_photos**
- jobs** 1—many **sms_logs**
- users** 1—many **jobs** (as assigned technician)
- pricing** referenced by **agreements.price** at creation time (not a hard FK — just a default lookup)

Indexing Notes

- `customers.nic` and `customers.phone` — indexed (primary search keys)
- `agreements.agreement_no` — unique index (looked up directly by technicians)
- `jobs.scheduled_date` — indexed (calendar view + reminder cron query by date)
- `jobs.technician_id` — indexed (reports filter/group by technician)
- `jobs.admin_confirmed` — indexed (Job Complete Requests queue filters on this)
- `jobs.is_deleted` — indexed (Deleted Jobs view filters on this)

How Visit Scheduling Now Works (Revised)

1. At agreement creation: `normal_count` + `hp_count` = total visits for the year, spaced `period_days` apart
 2. `schedulerService.js` generates `(normal_count + hp_count)` rows in `jobs`, each `period_days` apart starting from `start_date`
 3. `service_type_used` is left NULL until the technician actually performs that visit and tags it Normal or H/P
 4. (Optional application-level check, not enforced at DB level) — once `normal_count` visits have been tagged 'normal' and `hp_count` visits tagged 'hp', no further visits should be created without a renewal
-

Database Setup Guide (MySQL) — AC Service Management System (Revised per Wireframes)

Actual SQL to set up the database, plus sample queries mapped to each real data flow. Updated for the 9-table schema (3 roles, expanded AC unit fields, Normal/H-P count + period model, completion approval, pricing table).

1. Create Database

```
CREATE DATABASE ac_service_system
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE ac_service_system;
```

`utf8mb4` is required (not plain `utf8`) to correctly store **Sinhala Unicode text** in the `route` field.

2. Create Tables

```
-- USERS (3 roles: admin, system_user, technician)
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(150) NOT NULL,
  phone VARCHAR(20),
  role ENUM('admin','system_user','technician') NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  active BOOLEAN DEFAULT TRUE,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- CUSTOMERS
CREATE TABLE customers (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(150) NOT NULL,
  phone VARCHAR(20) NOT NULL UNIQUE,
  nic VARCHAR(20) NOT NULL,
  address TEXT,
  route VARCHAR(255),
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_nic (nic),
  INDEX idx_phone (phone)
);

-- AC UNITS (expanded: model, brand, dual serials)
```

```

CREATE TABLE ac_units (
    id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT NOT NULL,
    model VARCHAR(100),
    brand VARCHAR(100),
    serial_indoor VARCHAR(100),
    serial_outdoor VARCHAR(100),
    install_notes TEXT,
    FOREIGN KEY (customer_id) REFERENCES customers(id)
);

-- PRICING (admin-managed default prices)
CREATE TABLE pricing (
    id INT AUTO_INCREMENT PRIMARY KEY,
    service_type ENUM('normal','hp') NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- AGREEMENTS (Normal/H-P count + period model, replaces single service_type)
CREATE TABLE agreements (
    id INT AUTO_INCREMENT PRIMARY KEY,
    agreement_no VARCHAR(20) NOT NULL UNIQUE,
    customer_id INT NOT NULL,
    ac_unit_id INT NOT NULL,
    normal_count INT NOT NULL DEFAULT 0,
    hp_count INT NOT NULL DEFAULT 0,
    period_days INT NOT NULL,
    price DECIMAL(10,2),
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    amount_paid DECIMAL(10,2) NOT NULL,
    status ENUM('active','expired','cancelled','renewed') DEFAULT 'active',
    parent_agreement_id INT NULL,
    created_by INT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES customers(id),
    FOREIGN KEY (ac_unit_id) REFERENCES ac_units(id),
    FOREIGN KEY (parent_agreement_id) REFERENCES agreements(id),
    FOREIGN KEY (created_by) REFERENCES users(id),
    INDEX idx_agreement_no (agreement_no)
);

-- JOBS (expanded: service_type_used, admin_confirmed, is_deleted, postpone_days,
cancel_reason, comments)
CREATE TABLE jobs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    agreement_id INT NOT NULL,
    scheduled_date DATE NOT NULL,
    status ENUM('scheduled','in_progress','completed','postponed','cancelled') DEFAULT
'scheduled',
    technician_id INT NULL,

```

```

service_type_used ENUM('normal','hp') NULL,
admin_confirmed BOOLEAN DEFAULT FALSE,
is_deleted BOOLEAN DEFAULT FALSE,
postponed_from DATE NULL,
postpone_days INT NULL,
postpone_reason TEXT NULL,
cancel_reason TEXT NULL,
comments TEXT NULL,
completed_at DATETIME NULL,
notes TEXT,
created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (agreement_id) REFERENCES agreements(id),
FOREIGN KEY (technician_id) REFERENCES users(id),
INDEX idx_scheduled_date (scheduled_date),
INDEX idx_technician (technician_id),
INDEX idx_admin_confirmed (admin_confirmed),
INDEX idx_is_deleted (is_deleted)
);

-- JOB PHOTOS
CREATE TABLE job_photos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    job_id INT NOT NULL,
    photo_path VARCHAR(255) NOT NULL,
    uploaded_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    uploaded_by INT,
    FOREIGN KEY (job_id) REFERENCES jobs(id),
    FOREIGN KEY (uploaded_by) REFERENCES users(id)
);

-- SMS LOGS
CREATE TABLE sms_logs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT NOT NULL,
    job_id INT NULL,
    template_type ENUM('activation','reminder','completion') NOT NULL,
    message TEXT NOT NULL,
    sent_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(50),
    FOREIGN KEY (customer_id) REFERENCES customers(id),
    FOREIGN KEY (job_id) REFERENCES jobs(id)
);

```

3. Create the Database User (cPanel)

Via cPanel → **MySQL Databases**:

1. Create database: `youruser_acservice`
2. Create user: `youruser_appuser`, with a strong password
3. Add user to database with **All Privileges**

Then in your `.env` / Node.js App environment variables:

```
DB_HOST=localhost
DB_USER=youruser_appuser
DB_PASS=your_strong_password
DB_NAME=youruser_acservice
```

4. Sample Queries — Mapped to Each Data Flow

Flow A — Registration → Activation

```
-- Check if customer already exists
SELECT * FROM customers WHERE nic = ? OR phone = ?;

-- Create new customer (if not found)
INSERT INTO customers (name, phone, nic, address, route)
VALUES (?, ?, ?, ?, ?);

-- Add the AC unit (with new expanded fields)
INSERT INTO ac_units (customer_id, model, brand, serial_indoor, serial_outdoor)
VALUES (?, ?, ?, ?, ?);

-- Look up default pricing for pre-fill
SELECT * FROM pricing WHERE service_type = ?;

-- Generate next AS- number (application-level, then insert)
SELECT agreement_no FROM agreements ORDER BY id DESC LIMIT 1;
-- e.g. last was AS-00042 -> next is AS-00043 (zero-padded, generated in
numberingService.js)

-- Create the agreement (Normal/H-P counts + period, not a single type)
INSERT INTO agreements
  (agreement_no, customer_id, ac_unit_id, normal_count, hp_count, period_days, price,
start_date, end_date, amount_paid, created_by)
VALUES
  (?, ?, ?, ?, ?, ?, ?, CURDATE(), DATE_ADD(CURDATE(), INTERVAL 1 YEAR), ?, ?);

-- Auto-generate all visits: (normal_count + hp_count) jobs, spaced period_days apart
-- Example: normal_count=2, hp_count=2, period_days=90 -> 4 jobs, 90 days apart
INSERT INTO jobs (agreement_id, scheduled_date) VALUES
  (?, DATE_ADD(CURDATE(), INTERVAL 90 DAY)),
  (?, DATE_ADD(CURDATE(), INTERVAL 180 DAY)),
  (?, DATE_ADD(CURDATE(), INTERVAL 270 DAY)),
  (?, DATE_ADD(CURDATE(), INTERVAL 360 DAY));

-- Log the activation SMS after sending via Text.lk
INSERT INTO sms_logs (customer_id, template_type, message, status)
VALUES (?, 'activation', ?, 'sent');
```

Flow B — Job Lifecycle (with completion approval + type tagging)

```
-- Calendar view: all jobs on a given day (with status-based color coding)
SELECT j.*, a.agreement_no, c.name, c.address, c.route
FROM jobs j
JOIN agreements a ON j.agreement_id = a.id
JOIN customers c ON a.customer_id = c.id
WHERE j.scheduled_date = ? AND j.is_deleted = FALSE;

-- Assign technician
UPDATE jobs SET technician_id = ? WHERE id = ?;

-- Daily reminder check (run by reminderCron.js)
SELECT j.*, c.phone, c.name
FROM jobs j
JOIN agreements a ON j.agreement_id = a.id
JOIN customers c ON a.customer_id = c.id
WHERE j.scheduled_date = DATE_ADD(CURDATE(), INTERVAL 1 DAY)
    AND j.status = 'scheduled';

-- Technician searches by AS- number
SELECT j.*, a.agreement_no, c.name, c.address, c.route
FROM jobs j
JOIN agreements a ON j.agreement_id = a.id
JOIN customers c ON a.customer_id = c.id
WHERE a.agreement_no = ?
ORDER BY j.scheduled_date DESC;

-- Mark job in progress
UPDATE jobs SET status = 'in_progress' WHERE id = ?;

-- Upload a photo
INSERT INTO job_photos (job_id, photo_path, uploaded_by)
VALUES (?, ?, ?);

-- Check current photo count before completion (must be 4-5)
SELECT COUNT(*) AS photo_count FROM job_photos WHERE job_id = ?;
-- Application logic: reject "Complete" action if photo_count < 4

-- Technician marks complete AND tags the visit type (Normal or H/P)
UPDATE jobs
SET status = 'completed', service_type_used = ?, completed_at = NOW(), admin_confirmed =
FALSE
WHERE id = ?;

-- Job Complete Requests queue (admin-facing) – jobs completed but not yet confirmed
SELECT j.*, a.agreement_no, c.name
FROM jobs j
JOIN agreements a ON j.agreement_id = a.id
JOIN customers c ON a.customer_id = c.id
WHERE j.status = 'completed' AND j.admin_confirmed = FALSE;
```

```
-- Admin confirms the completion (this is what finally triggers the Completion SMS)
```

```
UPDATE jobs SET admin_confirmed = TRUE WHERE id = ?;
```

```
INSERT INTO sms_logs (customer_id, job_id, template_type, message, status)
VALUES (?, ?, 'completion', ?, 'sent');
```

```
-- Postpone a job (with days + reason, per wireframe)
```

```
UPDATE jobs
```

```
SET postponed_from = scheduled_date,
    scheduled_date = DATE_ADD(scheduled_date, INTERVAL ? DAY),
    postpone_days = ?,
    postpone_reason = ?,
    status = 'postponed'
```

```
WHERE id = ?;
```

```
-- Cancel a job (with reason)
```

```
UPDATE jobs SET status = 'cancelled', cancel_reason = ? WHERE id = ?;
```

```
-- Soft-delete a job (mistake/duplicate – different from cancellation)
```

```
UPDATE jobs SET is_deleted = TRUE WHERE id = ?;
```

```
-- Deleted Jobs view (admin)
```

```
SELECT * FROM jobs WHERE is_deleted = TRUE;
```

```
-- Job Cancellations view (admin) – distinct from Deleted Jobs
```

```
SELECT * FROM jobs WHERE status = 'cancelled' AND is_deleted = FALSE;
```

Flow C — Renewal & Archive

```
-- Find agreements expiring soon (for renewal follow-up)
```

```
SELECT * FROM agreements WHERE end_date <= DATE_ADD(CURDATE(), INTERVAL 30 DAY) AND status =
'active';
```

```
-- Renew: create new agreement linked to the old one (same Normal/H-P/Period model, can be
edited)
```

```
INSERT INTO agreements
```

```
(agreement_no, customer_id, ac_unit_id, normal_count, hp_count, period_days, price,
start_date, end_date, amount_paid, status, parent_agreement_id, created_by)
```

```
VALUES
```

```
(?, ?, ?, ?, ?, ?, ?, CURDATE(), DATE_ADD(CURDATE(), INTERVAL 1 YEAR), ?, 'active', ?, ?);
```

```
-- Mark old agreement as renewed
```

```
UPDATE agreements SET status = 'renewed' WHERE id = ?;
```

```
-- Cancel/archive an entire agreement (soft-delete)
```

```
UPDATE agreements SET status = 'cancelled' WHERE id = ?;
```

```
-- View archive (cancelled agreements)
```

```
SELECT * FROM agreements WHERE status = 'cancelled';
```

Search & Customer History

```
-- Universal search by NIC – returns ALL linked agreements/ACs
SELECT a.*, ac.model, ac.brand, ac.serial_indoor, ac.serial_outdoor
FROM agreements a
JOIN customers c ON a.customer_id = c.id
JOIN ac_units ac ON a.ac_unit_id = ac.id
WHERE c.nic = ?
ORDER BY a.created_at DESC;

-- Same, by phone
SELECT a.*, ac.model, ac.brand
FROM agreements a
JOIN customers c ON a.customer_id = c.id
JOIN ac_units ac ON a.ac_unit_id = ac.id
WHERE c.phone = ?
ORDER BY a.created_at DESC;

-- Direct lookup by AS- number (also used to pre-fill the Renew AC screen)
SELECT a.*, c.name, c.phone, c.nic, c.address, c.route, ac.model, ac.brand,
ac.serial_indoor, ac.serial_outdoor
FROM agreements a
JOIN customers c ON a.customer_id = c.id
JOIN ac_units ac ON a.ac_unit_id = ac.id
WHERE a.agreement_no = ?;

-- Customer loyalty duration
SELECT name, nic, phone, created_at,
       TIMESTAMPDIFF(YEAR, created_at, CURDATE()) AS years_as_customer
FROM customers
WHERE id = ?;

-- Full renewal chain for one AC (walk parent_agreement_id backwards)
WITH RECURSIVE agreement_chain AS (
  SELECT * FROM agreements WHERE id = ?
  UNION ALL
  SELECT a.* FROM agreements a
  JOIN agreement_chain ac ON a.id = ac.parent_agreement_id
)
SELECT * FROM agreement_chain ORDER BY start_date;
```

Reporting

```
-- Technician job count for a date range (only counts admin-confirmed completions)
SELECT COUNT(*) AS completed_jobs
FROM jobs
WHERE technician_id = ?
      AND status = 'completed'
      AND admin_confirmed = TRUE
      AND completed_at BETWEEN ? AND ?;
```

```
-- Technician performance, grouped by day (for a month view)
SELECT DATE(completed_at) AS day, COUNT(*) AS jobs_done
FROM jobs
WHERE technician_id = ?
      AND status = 'completed'
      AND admin_confirmed = TRUE
      AND completed_at BETWEEN ? AND ?
GROUP BY DATE(completed_at)
ORDER BY day;
```

Pricing (Admin-only)

```
-- View current pricing
SELECT * FROM pricing;

-- Update/add a price
INSERT INTO pricing (service_type, price) VALUES (?, ?)
ON DUPLICATE KEY UPDATE price = VALUES(price), updated_at = NOW();
```

User Management (Admin-only)

```
-- List all users
SELECT id, name, phone, role, active FROM users;

-- Add a new user (system_user or technician)
INSERT INTO users (name, phone, role, password_hash) VALUES (?, ?, ?, ?);

-- Update a user
UPDATE users SET name = ?, phone = ?, role = ? WHERE id = ?;

-- Deactivate (soft-delete) a user rather than hard-deleting
UPDATE users SET active = FALSE WHERE id = ?;
```

5. Backup Recommendation

UHL's Node.js/Python Silver plan includes **daily automated backups** — no extra setup needed. For an additional manual safety net:

```
0 2 * * * mysqldump -u youruser_appuser -pYOURPASS youruser_acservice >
/home/youruser/backups/db_$(date +%F).sql
```

Implementation Plan — Step by Step

Granular build plan, broken into 10 phases with individual tasks. Follow in order — each phase builds on the previous one.

Phase 1 — Project Setup, Database Schema, Authentication

1. Initialize Node.js project (`npm init`), install Express, mysql2 (or Sequelize), bcrypt, jsonwebtoken, dotenv
2. Initialize React project (`client/`) via Vite or Create React App
3. Set up `.env` with DB credentials, JWT secret placeholder
4. Create MySQL database and run all `CREATE TABLE` statements, including the 3-role `users` enum and the new `pricing` table (see `07_Database_Setup_Guide_MySQL.md`)
5. Build `models/user.model.js` — basic CRUD for users, `models/pricing.model.js` — CRUD for default pricing
6. Build `POST /api/auth/login` — verify phone/password, bcrypt compare, issue JWT with `{ id, role }` (role is one of admin/system_user/technician)
7. Build `auth.middleware.js` — verifies JWT on protected routes
8. Build `role.middleware.js` — checks role matches required access level; **Admin-only guard** specifically for `/api/users/*` and `/api/pricing/*`
9. Seed one Admin user manually to enable first login
10. Build `AddUsers.jsx` (Admin only) — add/update/deactivate System User and Technician accounts
11. Build `AddPrice.jsx` (Admin only) — set default price per service type (Normal/H-P)
12. Frontend: build Login screen, store JWT (React state, not localStorage), attach token to API calls
13. **Checkpoint:** Admin, System User, and Technician can all log in and get correctly routed/restricted by role; Admin can manage users and pricing

Phase 2 — Customer & Agreement Registration

1. Build `models/customer.model.js` , `models/acUnit.model.js` (with `model/brand/serial_indoor/serial_outdoor`), `models/agreement.model.js` (with `normal_count/hp_count/period_days/price`)
2. Build `GET /api/customers/search` — search by NIC / phone / AS-
3. Build `POST /api/customers` — create customer, checking for existing NIC/phone first
4. Build `numberingService.js` — generates next `AS-` number
5. Build `POST /api/agreements` — creates agreement + AC unit link + calls numberingService; **pulls default price from `pricing` table**, editable before saving
6. Build `schedulerService.js` — generates `(normal_count + hp_count)` job rows, spaced `period_days` apart
7. Build `smsService.js` — Text.ik API wrapper; wire up the Activation SMS trigger on agreement creation
8. Frontend: build `CustomerSearch.jsx` , `CustomerProfile.jsx` , `NewAgreement.jsx` — form includes Model/Brand/Serial(in)/Serial(out), Normal count, H/P count, Period (30/60/90/120 preset buttons), Price, "+

Add AC" for multiple units

9. **Checkpoint:** Can register a new customer, create an agreement with a mix of Normal/H-P visits on a chosen period, get an `AS-` number, and see an activation SMS logged

Phase 3 — Calendar, Scheduling, Assignment, Postpone/Cancel

1. Build `GET /api/jobs?date=` — jobs for a specific day
2. Build `PATCH /api/jobs/:id/assign` — assign technician
3. Build `PATCH /api/jobs/:id/postpone` — takes number of days + reason, shifts `scheduled_date` forward
4. Build `PATCH /api/jobs/:id/cancel` — takes a reason, sets status `cancelled`
5. Build `PATCH /api/jobs/:id/soft-delete` — sets `is_deleted = TRUE` (mistake correction, distinct from cancel)
6. Frontend: build `Calendar.jsx` — month/year filter, status counters, color-coded day cells (● Active, ● Postponed, ● Complete)
7. Frontend: build `JobSlot.jsx` — job detail view with Postpone (days + reason), Cancel (reason), Print, Download PDF, Comments
8. Frontend: build `DeletedJobs.jsx` and `JobCancellations.jsx` as two separate admin views
9. Frontend: build technician assignment UI (dropdown/select on a job)
10. **Checkpoint:** Jobs appear color-coded on the calendar; admin can assign a technician, postpone with a reason, cancel with a reason, or soft-delete a mistaken entry — each tracked separately

Phase 4 — Technician Mobile Module

1. Build `GET /api/jobs/:as_number` — technician lookup by AS- number
2. Build `PATCH /api/jobs/:id/status` — in_progress / completed transitions; completed also requires `service_type_used` (Normal or H/P)
3. Install and configure `multer` for file uploads
4. Build `POST /api/jobs/:id/photos` — enforce **min 4, max 5** photos, 5MB each (check count + size server-side, not just frontend); block "Complete" if fewer than 4 photos exist
5. Build `GET /api/jobs/:id/photos/:photoId` — authenticated photo retrieval
6. Frontend: build `TodayJobs.jsx`, `JobSearch.jsx`, `JobDetail.jsx` with `<input type="file" capture="environment" multiple>` for camera access, a Normal/H-P selector, and a comments field
7. Mobile-first CSS — large touch targets, simple single-column layout
8. **Checkpoint:** Technician can log in on a phone, search a job, upload 4–5 photos, tag the visit type, add a comment, and mark it complete (which puts it into the Job Complete Requests queue, not instantly finalized)

Phase 5 — SMS Automation & Job Complete Requests

1. Finalize `smsService.js` template rendering (activation, reminder, completion)
2. Build `reminderCron.js` — queries jobs scheduled for tomorrow, sends reminder SMS, logs to `sms_logs`
3. Set up the cPanel cron job entry (daily, e.g. 8am) pointing to `reminderCron.js`
4. Build `GET /api/jobs/complete-requests` — lists jobs where `status='completed'` and `admin_confirmed=FALSE`
5. Build `PATCH /api/jobs/:id/confirm` — Admin/System User confirms completion, sets `admin_confirmed=TRUE`, **this triggers the Completion SMS** (not the technician's initial "Complete" tap)

6. Frontend: build `JobCompleteRequests.jsx` — review queue showing photos/comments, with a Confirm action
7. Build `GET /api/sms/templates` and a simple admin UI to edit template wording
8. Build `GET /api/sms/logs` — admin-facing SMS history view
9. **Checkpoint:** Technician completions land in the Job Complete Requests queue; confirming one fires the Completion SMS and logs it; reminder cron runs on schedule

Phase 6 — Job Card Print & PDF View

1. Build `GET /api/jobs/:id/print` — returns print-friendly HTML (or a printable React view)
2. Add a **Download PDF** option alongside Print (e.g. render to PDF server-side, or use the browser's print-to-PDF as an interim solution)
3. Style for clean printing (hide nav/buttons in print CSS via `@media print`)
4. Once client provides their exact job card template, adjust layout to match
5. **Checkpoint:** A job card can be opened, printed, and downloaded as a PDF cleanly

Phase 7 — Archive, Renewal, Deleted Jobs, Job Cancellations

1. Build `POST /api/agreements/:id/cancel` — soft-delete, sets status to cancelled
2. Build `GET /api/agreements/archived` — lists cancelled agreements
3. Build `GET /api/agreements/search` (by ID/AS-/Phone) — powers the "Renew AC" search step
4. Build `POST /api/agreements/:id/renew` — new AS- number, `parent_agreement_id` link, old marked `renewed`; pre-fills Model/Brand/Serials/Normal/H-P/Period/Price from the existing agreement for editing
5. Frontend: build `Archive.jsx`, `RenewAgreement.jsx` (search → reuses the same form layout as New Agreement, pre-filled)
6. Confirm `DeletedJobs.jsx` and `JobCancellations.jsx` (built in Phase 3) are fully wired to their respective queries
7. **Checkpoint:** Cancelling moves an agreement to Archive (not deleted); renewing creates a fresh AS- linked to history, pre-filled from the old record; Deleted Jobs and Job Cancellations show correctly as separate lists

Phase 8 — Reporting Dashboard

1. Build `GET /api/reports/technician/:id?range=day|week|month` — counts only **admin-confirmed** completions
2. Frontend: build `Reports.jsx` — technician selector + date range + job count display (table or simple chart)
3. **Checkpoint:** Admin can view how many jobs each technician actually had confirmed-complete over a chosen period

Phase 9 — Deployment to cPanel

1. Confirm/activate **Setup Node.js App** in cPanel (Node version, app root, startup file `app.js`)
2. Create the MySQL database + user via cPanel's MySQL Databases tool
3. Set environment variables in the Node.js App interface (DB creds, JWT secret, Text.Ik API key)
4. Push code via cPanel Git Version Control (or SFTP), run **NPM Install**, click **Restart**
5. Build React locally (`npm run build`), upload `build/` into the server's `client/build` path

6. Set up the daily cron job for `reminderCron.js`
7. Test via cPanel's temporary URL (domain not purchased yet at this point)
8. **Checkpoint:** Full app is live and functional on the temporary hosting URL

Phase 10 — Domain Go-Live & UAT

1. Purchase the `.lk` domain (~4,500–5,000 LKR/year)
 2. Point domain to hosting account (nameservers or A record)
 3. Attach domain in cPanel, update Node.js App's Application URL
 4. Confirm free Let's Encrypt SSL is issued and active
 5. Run full UAT with the client using real wireframes (once provided) and their exact job card template
 6. Finalize SMS template wording with the client
 7. Fix any issues found during UAT
 8. **Go live**
-

Suggested Order of Priority If Timeline Is Tight

If phases need to be compressed, this is the minimum viable sequence to get a usable system in front of the client fastest:

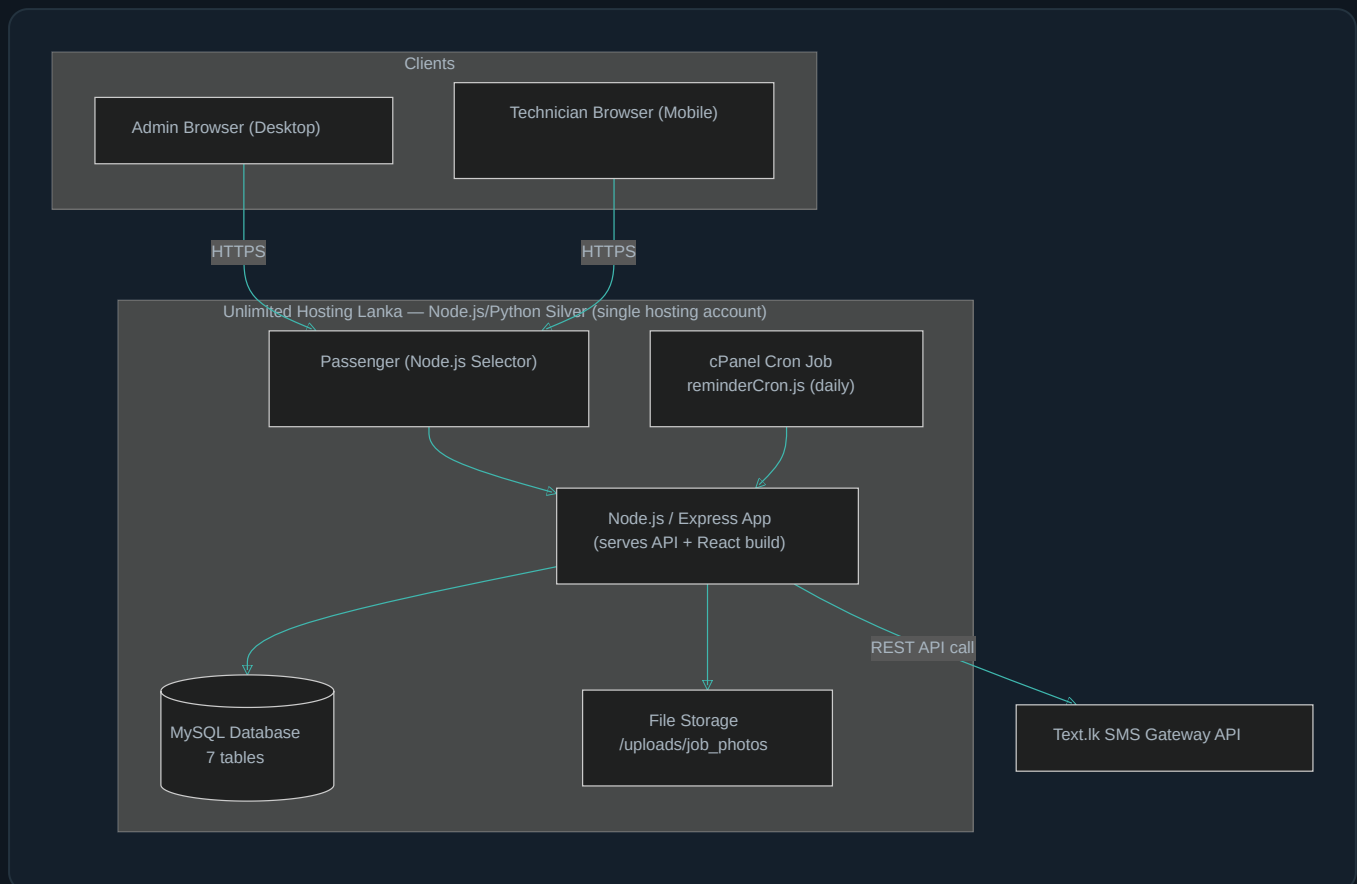
1. Phase 1 (auth) → 2. Phase 2 (registration + AS-) → 3. Phase 3 (calendar) → 4. Phase 4 (technician + photos) → then SMS, print, archive, reporting can follow in any order since they don't block each other.
-

Diagrams — AC Service Management System (Revised per Wireframes)

Updated after reviewing the client's hand-drawn wireframes: 3 roles, Normal/H-P visit-type tagging, completion approval flow, and status color-coding are now reflected below.

All diagrams in one place, written in **Mermaid** syntax — renders natively in GitHub, GitLab, Obsidian, VS Code (with Mermaid extension), and most modern markdown viewers.

1. System Architecture (Backend → Frontend → Hosting)



2. Entity Relationship Diagram (Database) — Revised

CUSTOMERS		
int	id	PK
varchar	name	
varchar	phone	UK
varchar	nic	
text	address	
varchar	route	

PRICING		
enum	service_type	normal hp
decimal	price	

AC_UNITS		
int	id	PK
varchar	model	
varchar	brand	
varchar	serial_indoor	
varchar	serial_outdoor	

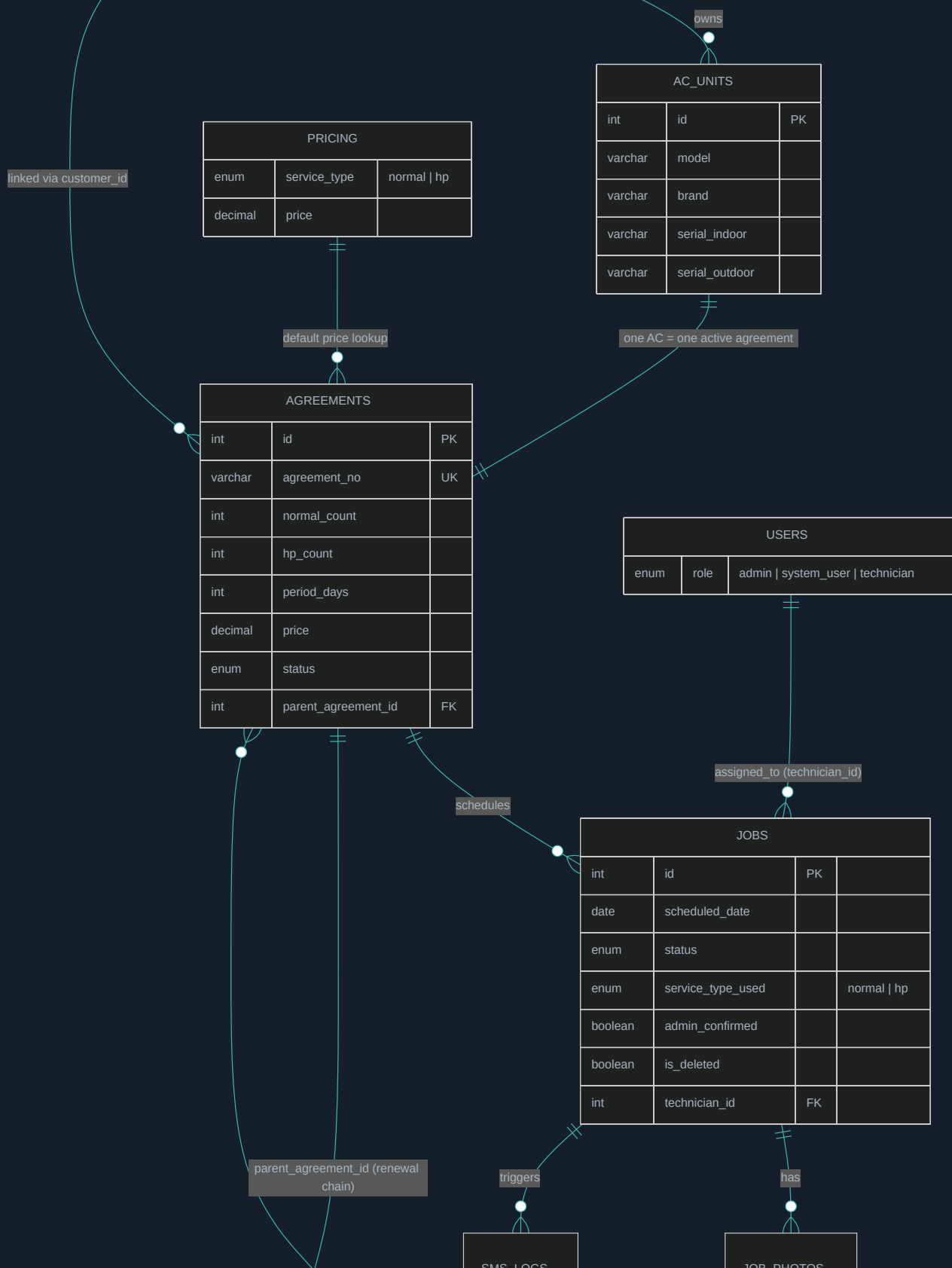
USERS		
enum	role	admin system_user technician

AGREEMENTS		
int	id	PK
varchar	agreement_no	UK
int	normal_count	
int	hp_count	
int	period_days	
decimal	price	
enum	status	
int	parent_agreement_id	FK

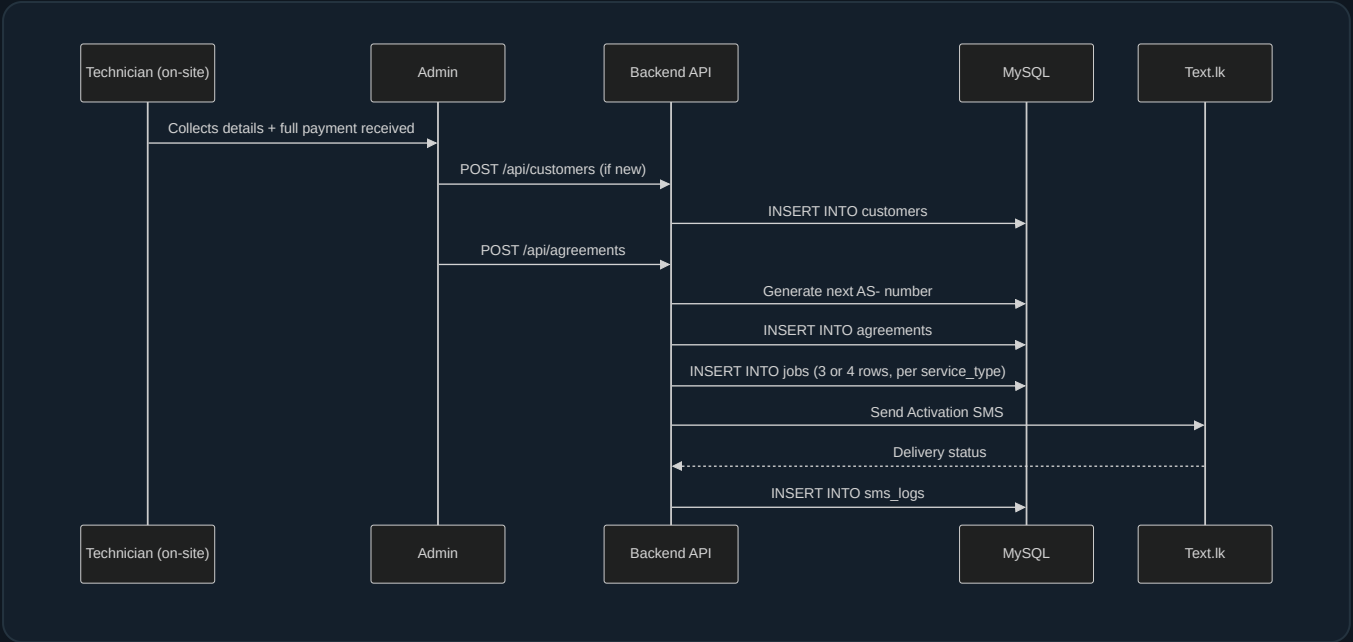
JOBS			
int	id	PK	
date	scheduled_date		
enum	status		
enum	service_type_used		normal hp
boolean	admin_confirmed		
boolean	is_deleted		
int	technician_id	FK	

SMS_LOGS	
----------	--

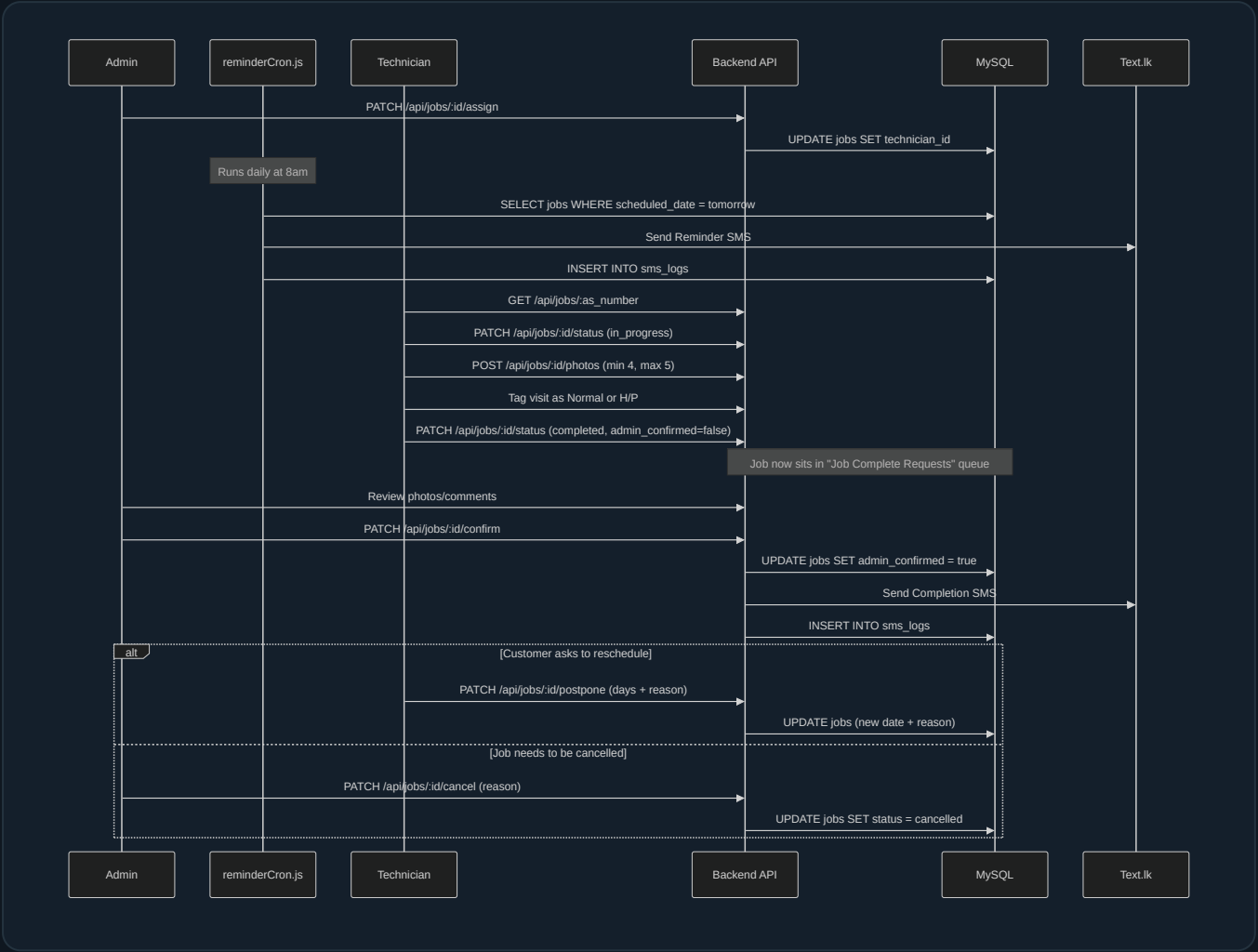
JOB_PHOTOS	
------------	--



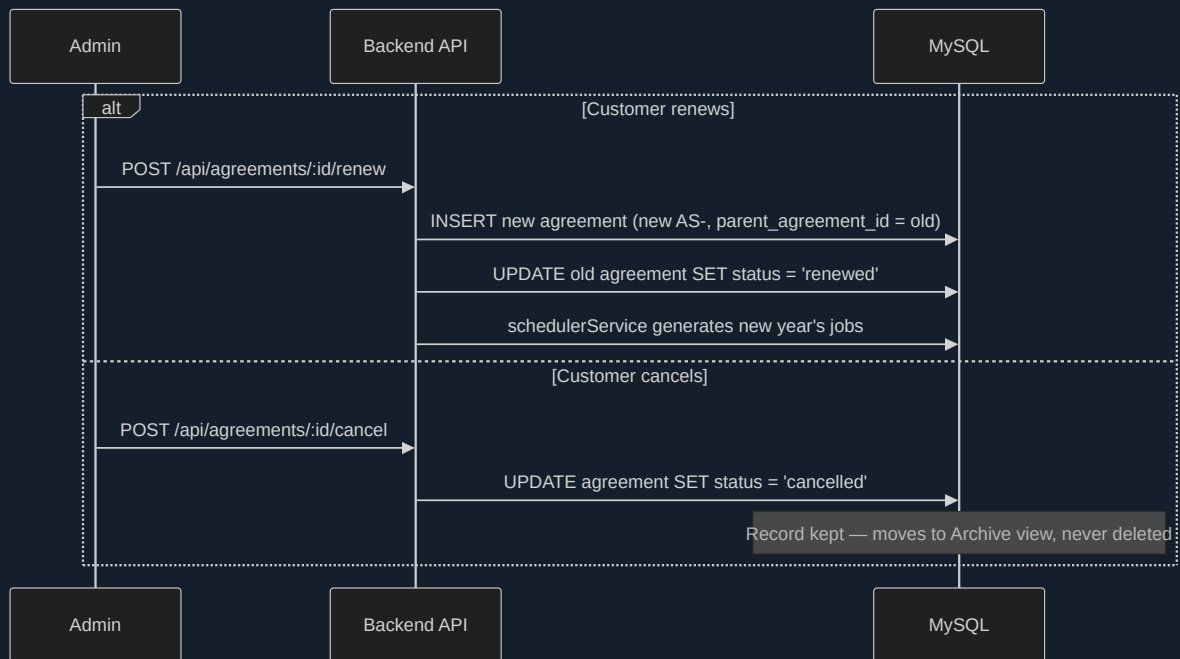
3. Data Flow A — Registration → Activation



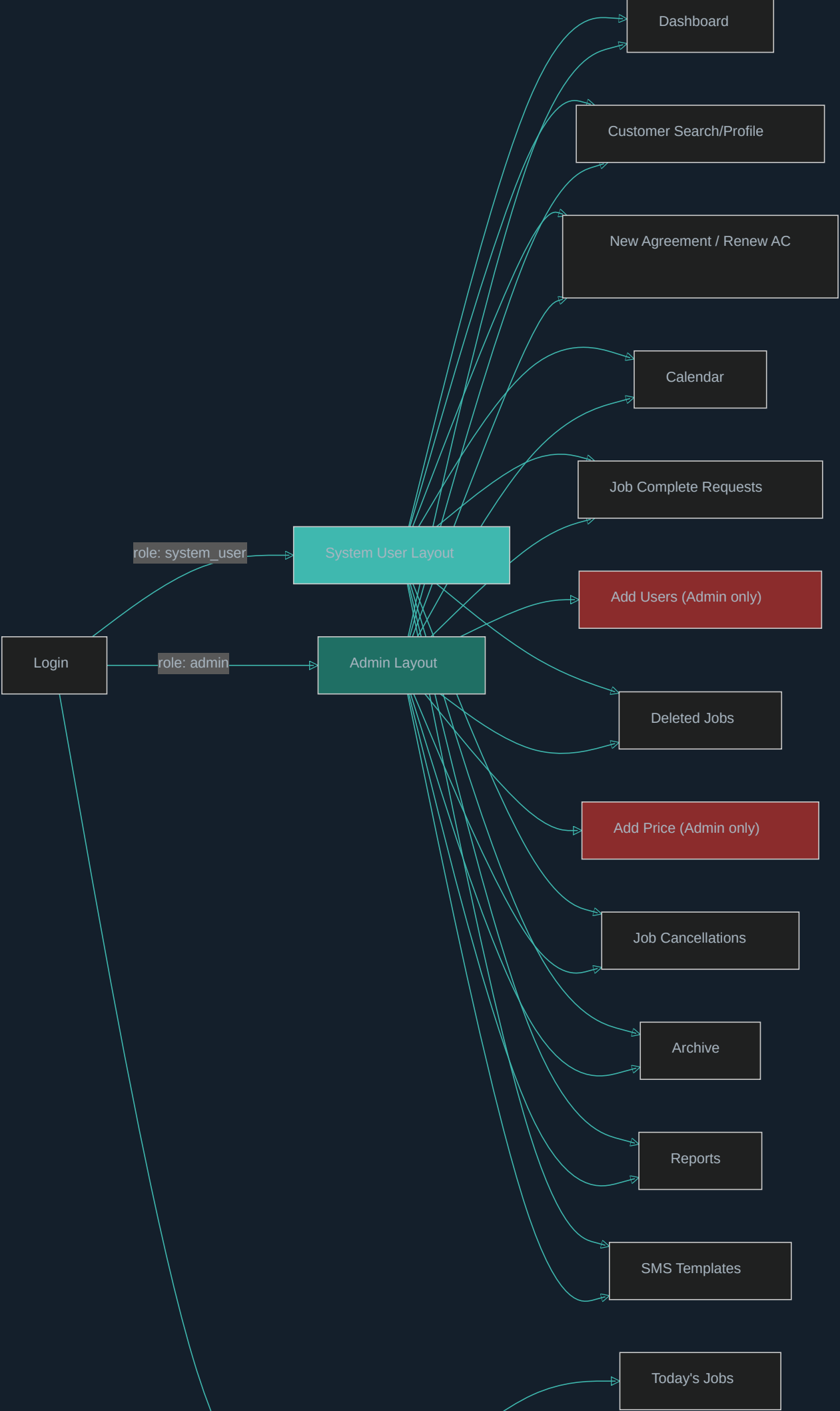
4. Data Flow B — Job Lifecycle (Revised: type-tagging + completion approval)



5. Data Flow C — Renewal & Archive



6. Role & Access Diagram (Revised — 3 roles)





7. URL / Routing Structure (Revised — entire system under /admin)

Root domain reserved for a future company showcase site (not built yet). The whole system — Admin and Technician sections both — sits under a single `/admin` base path.

yourdomain.lk/

Reserved — future showcase
site (not built yet)

Login Page

/admin/dashboard

/admin/customers

/admin/calendar

/admin/archive

/admin/reports

yourdomain.lk/admin/

/admin/job-complete-requests

/admin/deleted-jobs

/admin/cancellations

/admin/users (Admin only)

/admin/pricing (Admin only)

/admin/technician/*

/admin/technician/jobs

/admin/technician/job/:asNumber

8. Deployment / Request Routing on cPanel (Revised)



Tech Stack — Confirmed & Why It Aligns with Node.js + MySQL

1. Confirmed Stack

LAYER	TECHNOLOGY	STATUS
Frontend	React	Confirmed
Backend	Node.js + Express	Confirmed
Database	MySQL	Confirmed
Hosting	Unlimited Hosting Lanka — Node.js/Python Silver (cPanel)	Confirmed
SMS Gateway	Text.lk	Confirmed
File Storage	Server disk (within hosting quota)	Confirmed

2. Why This Stack Fits Together

Node.js + MySQL is one of the most common, well-supported pairings in web development — this isn't a compromise combination, it's a standard, proven one:

- `mysql2` or **Sequelize/Knex** (ORMs) connect Node directly to MySQL with mature, well-documented libraries
- Both are free, open-source, with large communities — easy to hire for, easy to find documentation/support for
- MySQL is included **free** with virtually every cPanel hosting plan (including this one) — no separate database hosting cost
- Node.js is explicitly supported on the chosen hosting tier (UHL's Node.js/Python Silver plan uses CloudLinux's Node.js Selector + Passenger)

Why not a different database (e.g. MongoDB)? This system is fundamentally **relational** — a Customer has many Agreements, an Agreement has many Jobs, a Job has many Photos, and there's a self-referencing renewal chain (`parent_agreement_id`). This is exactly the kind of structured, foreign-key-heavy data MySQL is built for. A NoSQL database would add complexity here for no benefit.

Why React specifically?

- One codebase serves both the admin (desktop) and technician (mobile) interfaces — fully responsive via CSS, no separate mobile app needed
- Wide talent pool, easy to maintain long-term
- Component-based structure matches this system's screen breakdown (Dashboard, Calendar, JobDetail, etc.) naturally

Why Express specifically (not Fastify, Koa, NestJS, etc.)?

- Simplest, most widely-documented Node framework — appropriate for a REST API of this size (~22 endpoints after adding user management and pricing routes per the wireframe review)

- Lower learning curve, faster to build and maintain than a more opinionated framework like NestJS, without sacrificing capability for this project's scope

3. How Everything Runs Together (Summary)



One process, one server, one hosting bill — this is the direct result of choosing a stack that's fully compatible with what a single cPanel Node.js hosting account can run.

4. Cost Impact of This Stack Choice

ITEM	COST
React, Node.js, Express, MySQL	Free — all open source, no licensing fees
ORM (Sequelize/Knex) if used	Free — open source
Hosting that supports this exact stack	Already selected: ~18,500 LKR/year (Node.js/Python Silver)

No part of this stack requires a paid license, a separate paid service, or specialized/expensive hosting — it was chosen specifically because it's the most cost-friendly path that still matches the team's existing Node.js/Python familiarity.

5. What Would Have Changed With a Different Hosting Answer

Noted for completeness — if UHL's cPanel had **not** supported Node.js, the fallback plan was:

LAYER	FALLBACK
Backend	PHP + Laravel (instead of Node/Express)
Database	MySQL (unchanged — Laravel pairs with MySQL just as naturally)
Hosting	Regular "Unlimited Hosting" plan (~7,500 LKR/year, cheaper)

This was **not** needed since Node.js support was confirmed on the Silver tier — but it's documented here in case hosting circumstances change in the future.

MEMORY — AC Service Management System

Condensed, structured project memory. Purpose: if this conversation is picked up again later (new chat, new team member, or Claude with no context), this file alone should be enough to get back up to speed instantly.

Last updated: after reviewing the client's 8 hand-drawn wireframe photos — several structural changes below.

Project

Web-based **AC Service Management System** for an AC servicing company. Manages 1-year service agreements, technician scheduling, automated SMS, photo-proof job execution, renewals, and reporting.

Confirmed Stack

- Frontend: **React**
- Backend: **Node.js + Express**
- Database: **MySQL**
- Hosting: **Unlimited Hosting Lanka — Node.js/Python Silver plan** (~18,500 LKR/year, 10GB NVMe SSD, 10 MySQL DBs)
- SMS: **Text.lk** (confirmed twice — pay-as-you-go, no setup fee, Rs 0.64–0.84/SMS)
- Deployment: single Express app serves both API and React static build — no separate frontend hosting

URL Structure — REVISED (root path reserved for future showcase site)

The root domain (`yourdomain.lk/`) is **reserved for a future company showcase/marketing site** — not part of this system, to be built at a later stage (not now). **The entire system — both Admin and Technician sections — is mounted under a single `/admin` base path** instead of splitting across `/admin` and `/technician` at the root level. React Router uses `basename="/admin"` ; Express serves the React build only under `/admin/*` (`app.use('/admin', express.static(...)) + app.get('/admin/*', ...)`), leaving `/` free for the future site (currently a placeholder or redirect to `/admin`).

Example paths: `yourdomain.lk/admin/` (login), `/admin/dashboard` , `/admin/customers` , `/admin/calendar` , `/admin/technician/jobs` , `/admin/technician/job/:asNumber` .

Domain Status

Not yet purchased. Will be bought (~4,500–5,000 LKR/year) **after** development is complete, just before go-live.

Roles — REVISED: now 3, not 2

- **Admin** — total authority: everything + user management (add/update/delete users) + pricing management
- **System User** (office staff) — day-to-day ops: customers, agreements, jobs, technician assignment, renewals, cancellations, reports. **No** user management or pricing access
- **Technician** — own jobs only, individual login (not shared)

Service Model — REVISED (this is the biggest change)

Old: one fixed type per agreement — Normal (3 visits/year) OR Hybrid (4+ visits/year). **New (per wireframe, confirmed by client):** an agreement has **two separate counters** — `normal_count` and `hp_count` — plus a `period_days` field (30/60/90/120 preset options). Total visits scheduled = `normal_count + hp_count`, spaced `period_days` apart. Each individual visit is **not** pre-labeled — the **technician tags it Normal or H/P at execution time** (decrements the relevant count). Example: `normal_count=2`, `hp_count=2`, `period_days=90` → 4 visits scheduled 90 days apart; technician tags each one as they do it.

AC Unit Fields — EXPANDED

Now includes: **Model**, **Brand**, **Serial Number (Indoor unit)**, **Serial Number (Outdoor unit)** — not just a generic label. Split ACs have two physical units, both serials tracked separately.

Completion Flow — REVISED (new approval step)

Technician tapping "Complete" does **not** immediately finalize the job or send the Completion SMS. It sets `status='completed'` but `admin_confirmed=FALSE`, landing in a new **"Job Complete Requests"** queue. Admin/System User reviews and confirms (`admin_confirmed=TRUE`) — **this confirmation is what actually triggers the Completion SMS.**

Photo Upload — REVISED

Minimum 4 required, maximum 5, 5MB each (was previously "max 5" with no minimum).

Postpone / Cancel — EXPANDED

- **Postpone:** requires **number of days** + a reason (not just a new date)
- **Cancel:** requires a reason/comment, goes to a **Job Cancellations** view
- **Deleted Jobs** is a **separate** view from Job Cancellations — soft-deletes for mistake corrections (duplicate/wrong entry), distinct from a customer-driven cancellation

New Admin-Only Features (from wireframes)

- **Add Users** — Admin manages System User and Technician accounts (add/update/deactivate)
- **Add Price** — Admin manages default pricing per service type (Normal/H-P), pre-fills into new agreements (still editable per agreement)

New Admin Views (from wireframes)

- **Job Complete Requests** — approval queue (see Completion Flow above)
- **Deleted Jobs** — soft-deleted jobs (mistakes)
- **Job Cancellations** — customer-cancelled jobs (distinct from Deleted Jobs)

Calendar Status Color Coding (confirmed from wireframe)

● Red = Active/Scheduled · ● Orange = Postponed · ● Green = Complete

Job Card — EXPANDED

Print **and** Download PDF (was print-only before).

Renewal Flow — clarified

Search by ID/AS-/Phone ("Renew AC" screen) → loads the **same form** as New Agreement, pre-filled with existing AC/agreement data → admin adjusts Model/Brand/Serials/Normal count/H-P count/Period/Price as needed → "Renew" button → new **AS-** number generated, **parent_agreement_id** links to the old one, old marked **renewed** .

Confirmed Business Rules (unchanged from before)

- Data entered into system **only after full payment** is received
- Agreement number format: **AS-00001** — one per AC unit, serial, never reused (client's wireframe sketches showed a longer example number, but this wasn't confirmed as a required format — flagged as still-open, see below)
- Customer search keys: **NIC** (primary) and **Phone** (unique ID) — searching either shows ALL linked AC units/agreements
- Renewal → **new** AS- number issued, linked to old via **parent_agreement_id** , old marked **renewed** (never overwritten)
- Cancellation → soft-delete only, moves to Archive/Job Cancellations, **never** hard-deleted
- SMS fires at 3 points: **Activation** (on registration), **Reminder** (1 day before scheduled job — via daily cron), **Completion** (on **admin confirmation**, not technician tap)
- URL structure: **see "URL Structure — REVISED" section above** — root reserved for future showcase site, entire system now under **/admin**

Database — now 9 Tables (was 7)

customers , **ac_units** (expanded), **agreements** (revised), **jobs** (expanded), **job_photos** , **sms_logs** , **users** (3 roles), **pricing** (new). (Full schema: **06_Database_Design.md** . Setup SQL: **07_Database_Setup_Guide_MySQL.md** .)

Screens — now 14 Admin (was 9) + 4 Technician (unchanged)

Admin: Dashboard, Customer Search, Customer Profile, New Agreement, **Renew Agreement**, Calendar, **Job Slot (detail)**, Job Card (print+PDF), **Job Complete Requests**, **Deleted Jobs**, **Job Cancellations**, Archive, Reports, SMS Templates, **Add Users**, **Add Price** Technician: Login, Today's Jobs, Job Search, Job Detail (now includes photo min/max + Normal/H-P tagging + comments)

Build Plan — 10 Phases (updated task lists, same overall order)

1. DB schema + auth + **user mgmt** + **pricing** → 2. Customer/Agreement registration **with Normal/H-P/Period model** → 3. Calendar/scheduling/assignment/**postpone+cancel+deleted-jobs** → 4. Technician mobile module + photos (**min4/max5**) + **type tagging** → 5. SMS automation + **Job Complete Requests approval** → 6. Job card print + **PDF** → 7. Archive/renewal + **deleted jobs/cancellations views** → 8. Reporting (confirmed-only counts) → 9. Deploy to cPanel (temp URL) → 10. Domain purchase + go-live + UAT (Granular task breakdown: **08_Implementation_Plan_Step_By_Step.md**)

Budget (as last discussed — unaffected by wireframe review)

- One-time development fee: **not yet finalized** — market estimate range 350,000–1,200,000+ LKR (mid-estimate ~700,000–900,000 LKR for ~35–45 developer-days). Note: wireframe-driven scope additions (user mgmt, pricing, approval queue, extra views) may push actual effort toward the higher end of this range — not yet re-quoted.
- Recurring: ~18,500 LKR/year hosting + ~4,500–5,000 LKR/year domain (from Year 1, post-dev) + variable SMS cost

Documents That Exist

1. `AC_Service_Management_System_Plan.md` — original full plan (pre-wireframe)
2. `Reordered_Scenario_Flow.md` — client's raw scenario reorganized chronologically (pre-wireframe)
3. `System_Design.md` — original technical design doc (pre-wireframe)
4. `AC_System_Full_Design_Documentation.html` — consolidated HTML doc w/ diagrams (pre-wireframe)
5. `AC_System_Budget_Proposal.docx` — client-facing budget Word doc (pre-wireframe scope)
6. `01_Conversation_Summary.md` through `10_Tech_Stack.md` — markdown package (02, 05, 06, 07, 08, 09 now **revised** post-wireframe; 01, 03, 04, 10 mostly unaffected structurally but 04 folder structure updated with new screens/routes)
7. This file (`MEMORY.md`)

Note: the first 5 documents above (plan, scenario flow, system design, HTML doc, budget) were created **before** the wireframe review and have **not** been updated to reflect the role/service-model/schema changes below. Treat the numbered `0X_*.md` files as the current source of truth; the earlier documents are historical/pre-wireframe.

Still Open / Pending

- ☐ Final SMS template wording (activation, reminder, completion)
- ☐ Job card exact visual layout (wireframe gives structure/fields, final styling still TBD)
- ☐ Wireframes — received and incorporated (this update)
- ☐ Final development fee amount (client/team still deciding; scope grew since original estimate)
- ☐ AS- number format — wireframe examples showed a longer string (e.g. "AS260718..."); not yet confirmed whether this is a required date-encoded format or just how the client wrote an example. Current design defaults to simple `AS-00001` serial.

Key Corrections Made Across This Conversation (don't revert to earlier assumptions)

- ~~Domain already owned~~ → **Domain not yet purchased**, buy after dev, before go-live
- ~~Notify.lk as SMS gateway~~ → **Text.lk confirmed** instead
- ~~PHP as backend~~ → **Node.js/Express confirmed**
- ~~2 roles (Admin, Technician)~~ → **3 roles (Admin, System User, Technician)**
- ~~Single service_type per agreement (Normal or Hybrid)~~ → **normal_count + hp_count + period_days, mixed per agreement, tagged per visit**

- ~~Max 5 photos, no minimum~~ → **Min 4, max 5 photos**
 - ~~Technician "Complete" tap = instantly done + SMS sent~~ → **Goes to Job Complete Requests queue; Admin confirmation triggers SMS**
 - ~~Job card: print only~~ → **Print + Download PDF**
 - ~~One "Archive" view for everything cancelled~~ → **Archive (agreements) + Deleted Jobs + Job Cancellations are three distinct views**
 - ~~Root domain (/) serves the system's login page, /technician at root level~~ → **Root reserved for a future showcase site; entire system (Admin + Technician) moved under /admin**
-